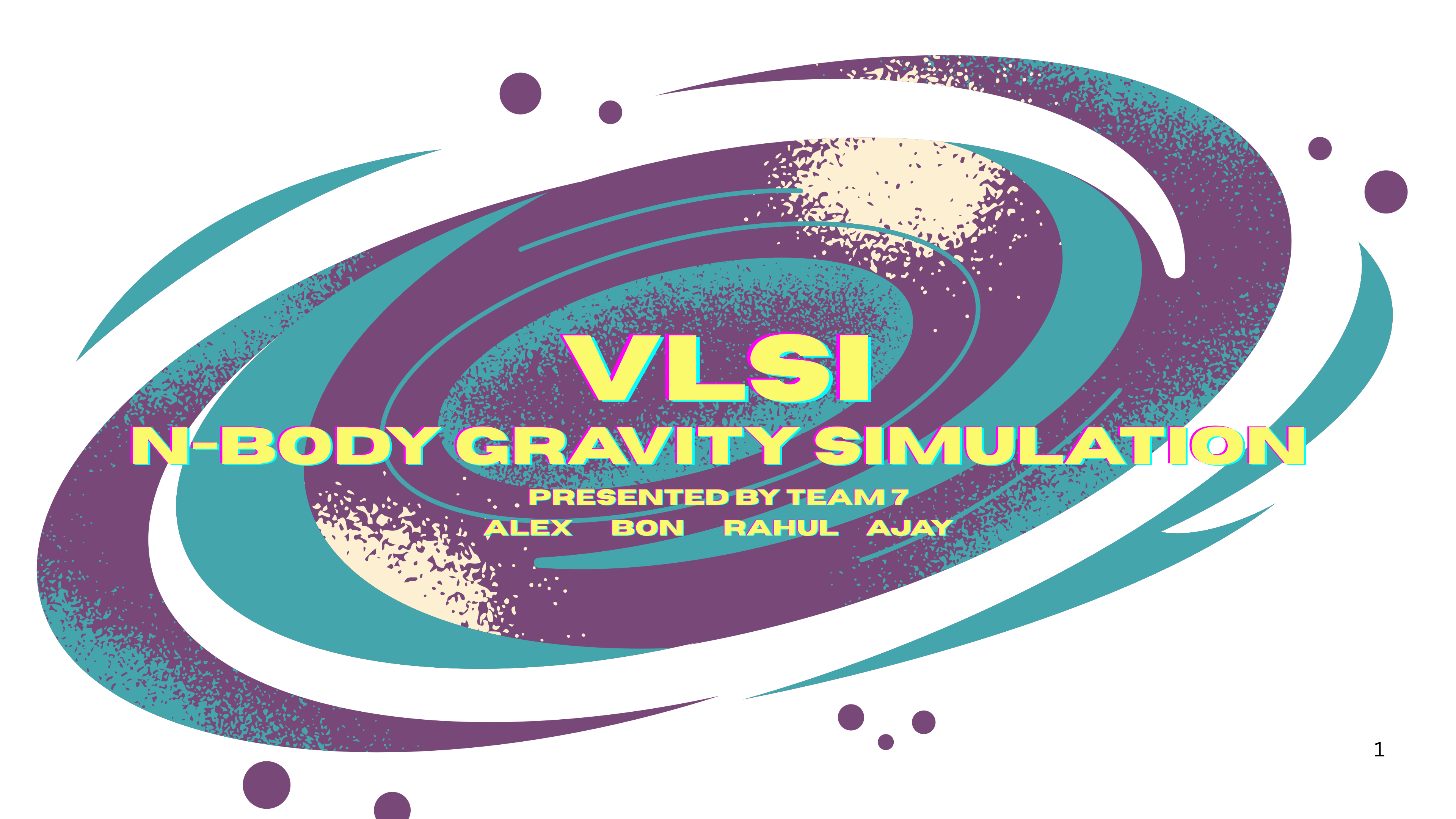




VLSI

N-BODY GRAVITY SIMULATION

PRESENTED BY TEAM 7

ALEX BON RAHUL AJAY

Background

Why N-body Simulation

Project Requirements:

- Scope allowing for focus on good hardware design
- Solve something visually interesting / fun (for demo)
- Takes advantage of hardware acceleration better than FPGA



Why N-body Simulation

Graphics Pipeline:

- Visually cool
- Expandable scope:
 - rasterizer → texture map → multiple stages / pipes → lighting
- Potential for good hardware design

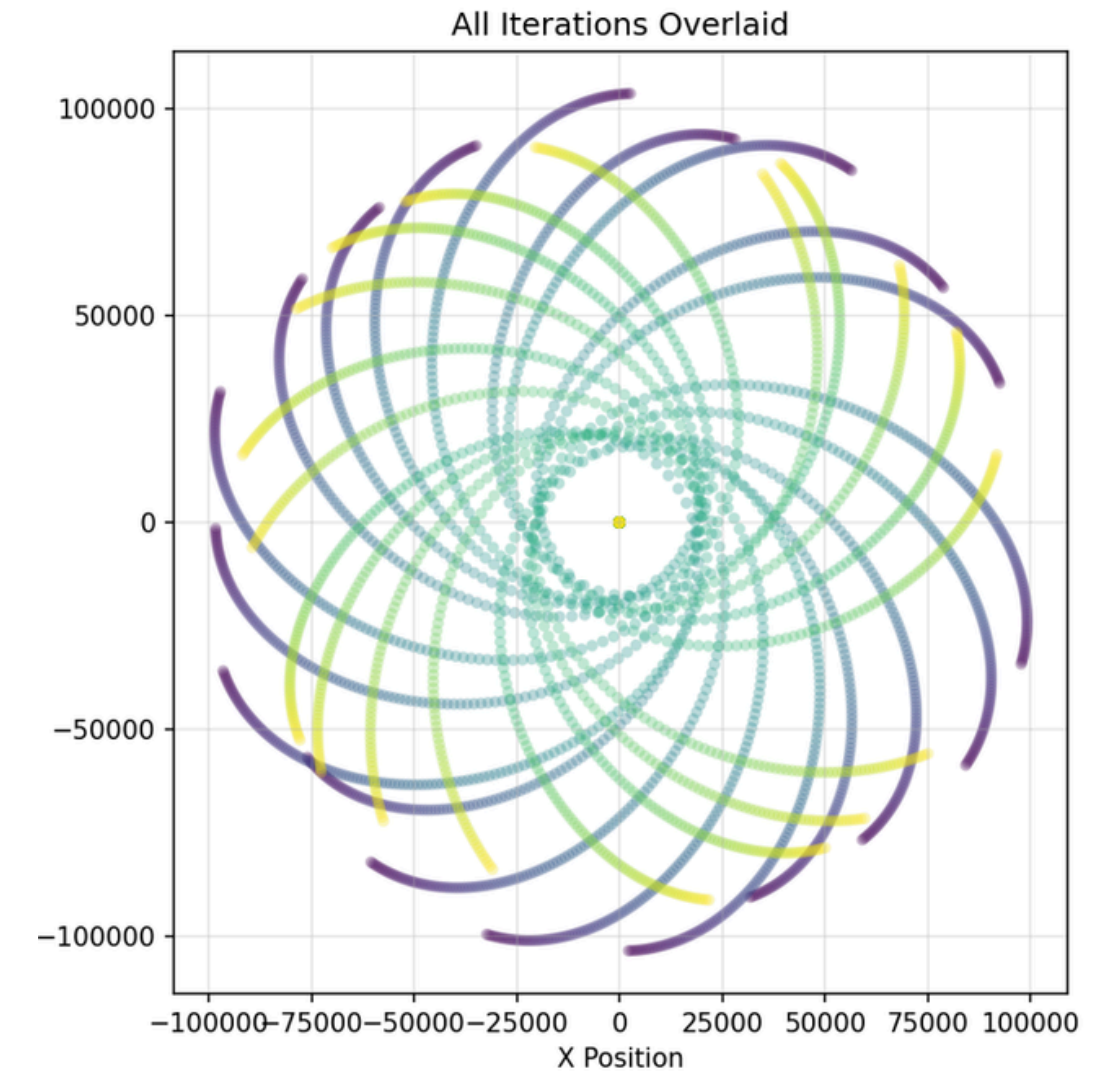


But:

- Requires a good amount of background knowledge
- Complex software to make use of hardware (EG: rest of graphics pipeline)

Why N-body Simulation

- Visually interesting output
- Relatively simple to use
- Simple concept for easy onboarding
- Scope for good hardware design
- Good compute / IO ratio



Background



$$\mathbf{F}_i = G \sum_{\substack{j=1 \\ j \neq i}}^N \frac{m_i m_j (\mathbf{r}_j - \mathbf{r}_i)}{\|\mathbf{r}_j - \mathbf{r}_i\|^3}$$

$$\mathbf{a}_i = \frac{\mathbf{F}_i}{m_i}$$

$$\begin{aligned} \mathbf{v}_i^{(k+1)} &= \mathbf{v}_i^{(k)} + \mathbf{a}_i^{(k)} \Delta t, \\ \mathbf{r}_i^{(k+1)} &= \mathbf{r}_i^{(k)} + \mathbf{v}_i^{(k+1)} \Delta t. \end{aligned}$$

- System driven by effect of gravitational forces between particles
- No general closed form solution exists - ours simulates evolution in finite timesteps
- Tiny differences in initial conditions get amplified to vastly different outcomes given enough time
- Each particle exerts a force on every other particle: $O(n^2)$ complexity

Design Goals



Design Constraints

- 25 GPIO Pins
- $850\text{ }\mu\text{m} \times 850\text{ }\mu\text{m}$ Area
- 400 MHz Max
- Need to run cool looking simulation
- Memory bandwidth constraints



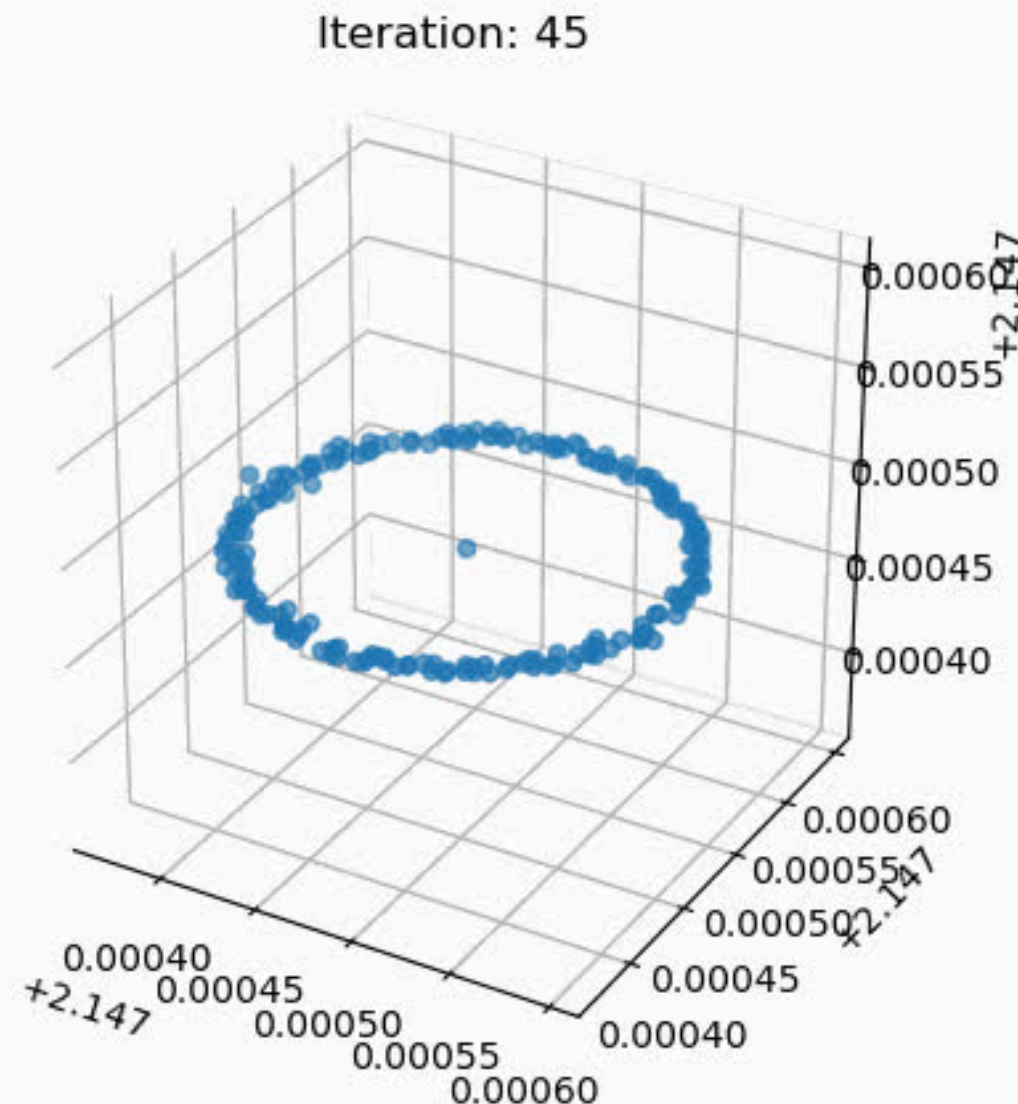
High level design decisions

- Make use of I/O:
 - Minimize unused I/O time-slots for high bandwidth data (output tap)
 - Don't waste pins on low bandwidth I/O (JTAG control chain)
- Minimize area:
 - Fixed point (still can look nice)
 - Optimize particle datatype (bits per field)
- High clock frequency:
 - Pipeline heavily
- Reduce memory bandwidth bottleneck:
 - Avoid repeated reads of same information where avoidable
 - Avoid separate pass pass for sim state outputting

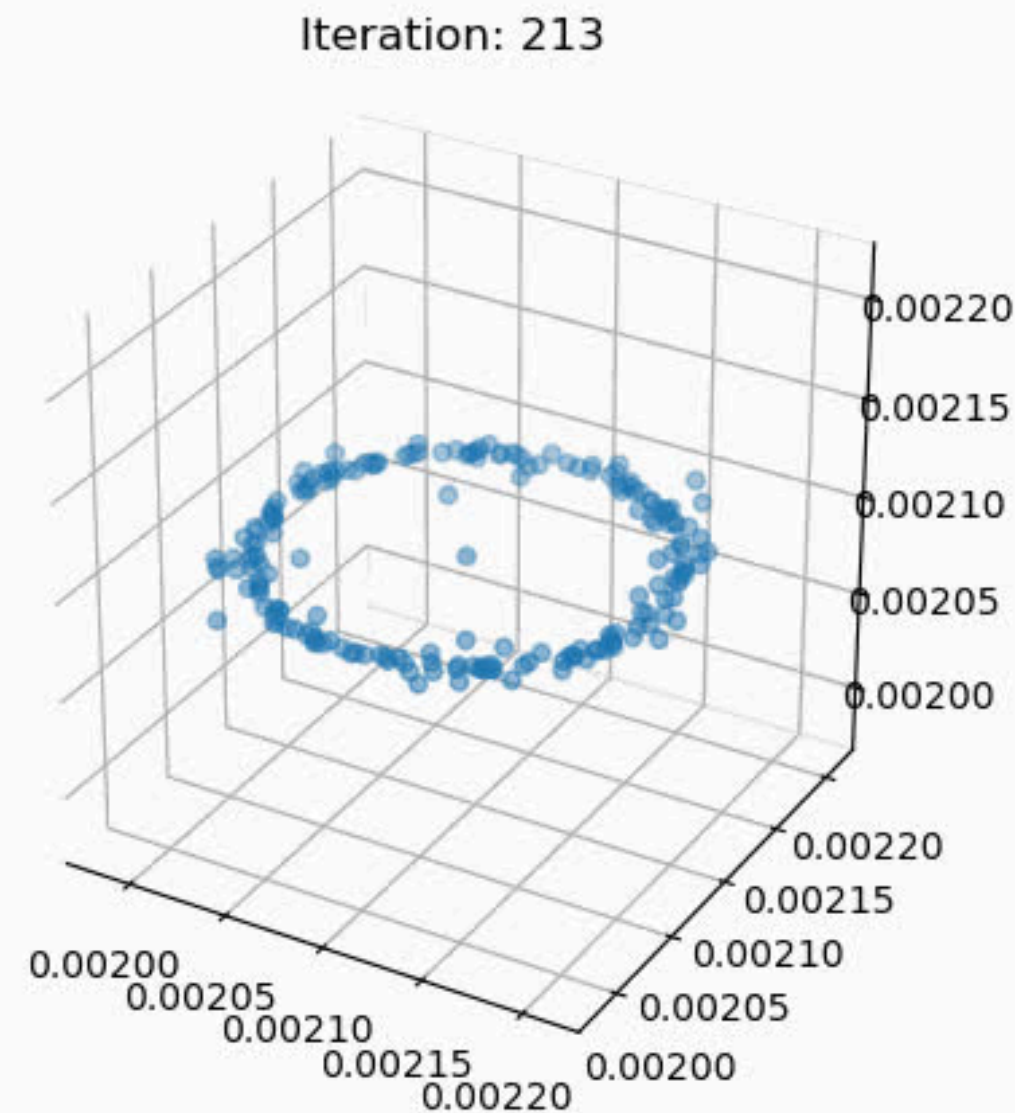
Fixed Point Bitwidth Simulation

- Simulation state represented in fixed point
- C++ simulation used to find appropriate bitwidths and good initial conditions
- Various different simulation states found - most can be found as bitwidths for g and m can be changed to accomodate a range of values

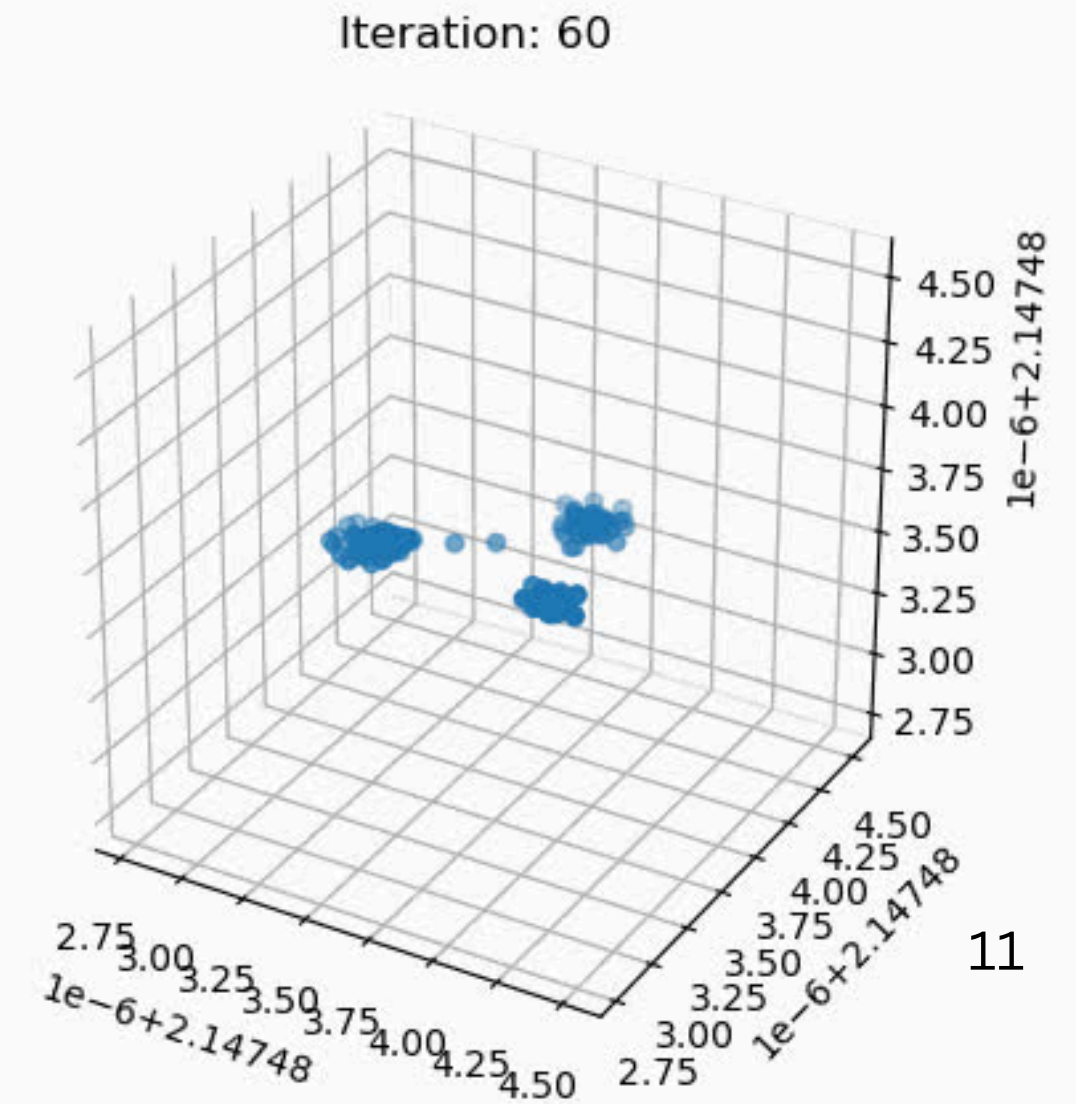
Light Center Mass



Heavy Center Mass

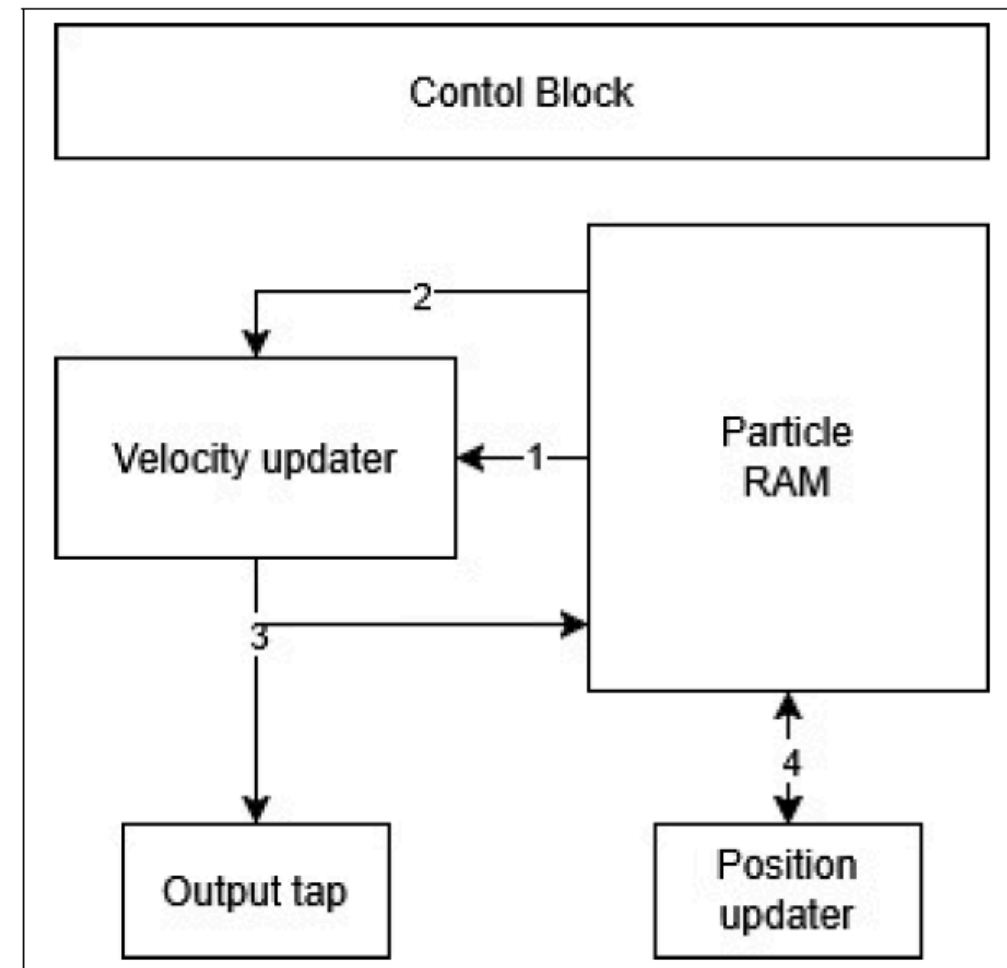


No Center Mass



Architecture

System Architecture Intro

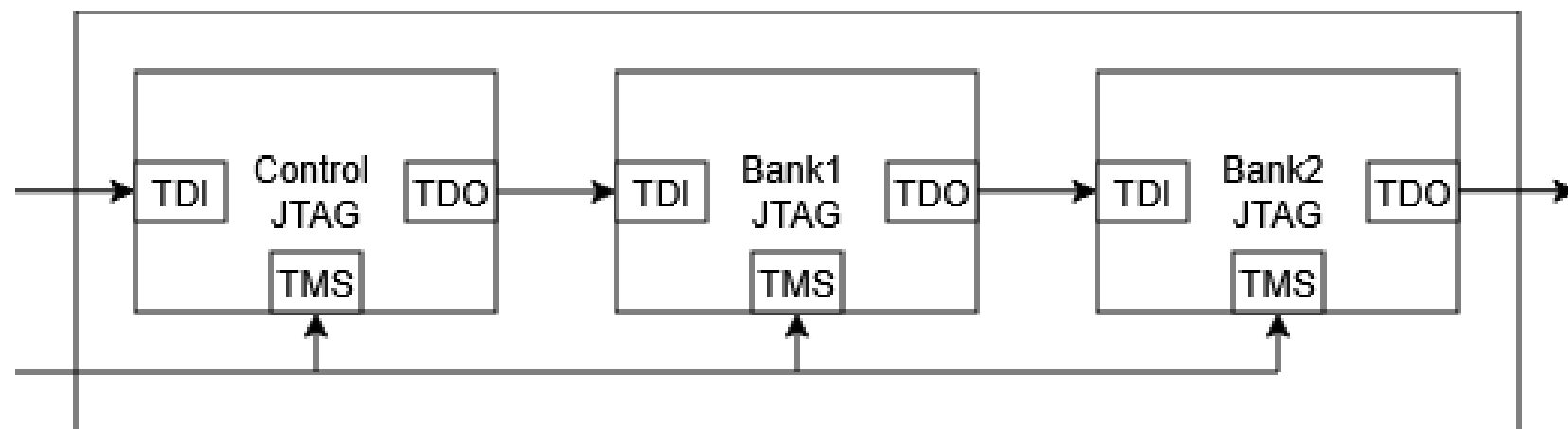


Control block diagram of the N-Body Accelerator

1. Load a particle into the velocity updater.
2. Stream all other particles through the velocity updater to apply all accelerations to the loaded particles
3. Write the accelerated particle back to the particle SRAM and the output tap. If some particles are yet to be accelerated go back to step 1 for the next particle, else continue on to step 4.
4. Apply the velocities to each particle to find their next position

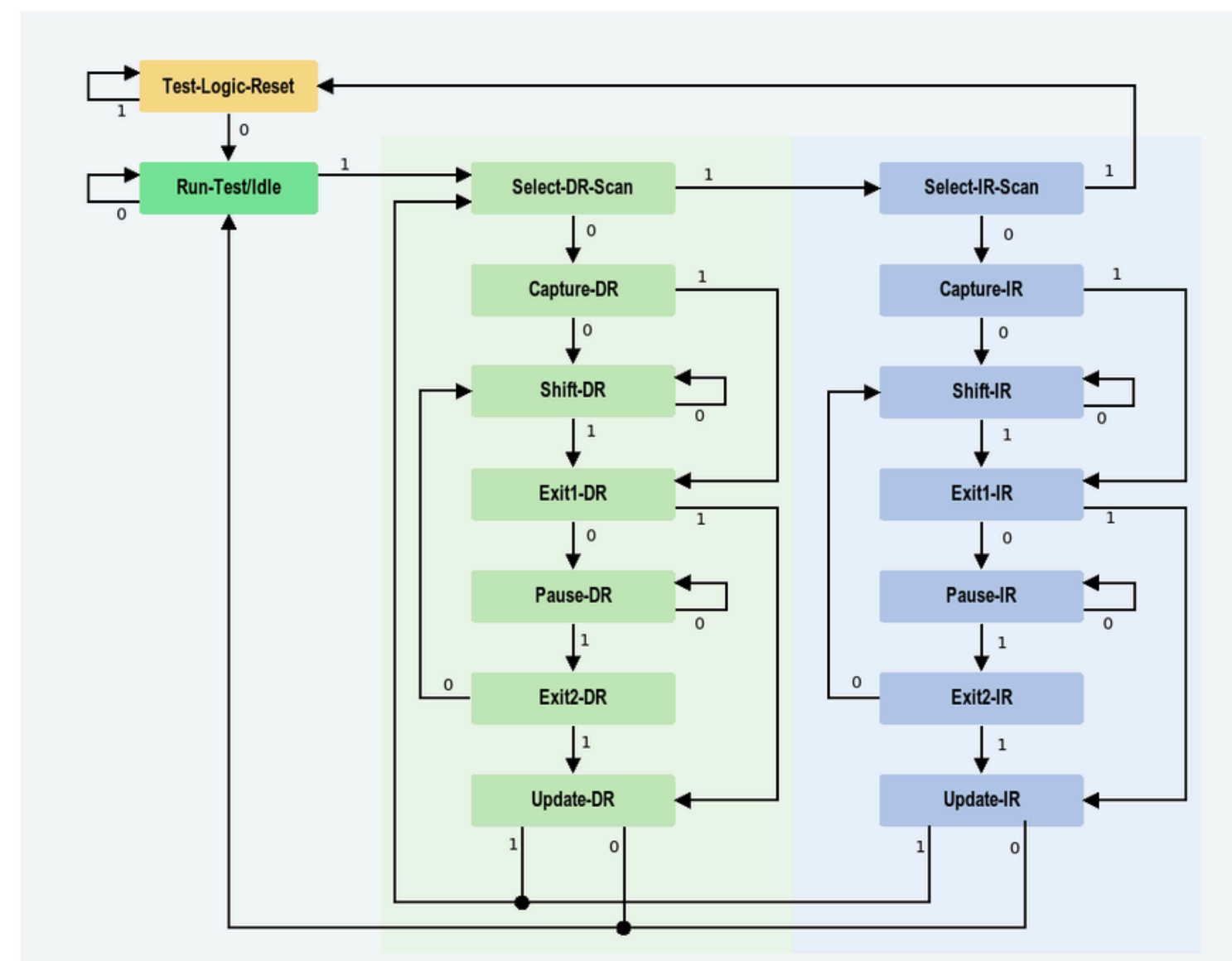
The position update is purely combinatorial. It reads / decodes the current particle being read from the SRAM, and applies the velocity ready to be written back in the next cycle. In the event that the article goes out of bounds, it simply wraps round back round.

Runing a simulation



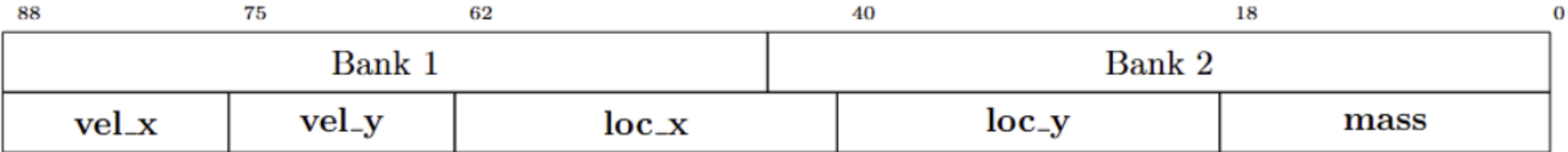
Instruction	Binary	Description	DR width used
IR_NOP	0000	No operation	0
IR_RUN_EPOCHS	0011	Set number of simulation epochs	32
IR_GET_EPOCHS	0100	Read remaining epochs	32
IR_SET_G	0101	Write gravitational constant	32
IR_GET_G	0110	Read gravitational constant	32
IR_SET_NUM_PARTICLES	0111	Set number of particles	12
IR_GET_NUM_PARTICLES	1000	Read number of particles	12

Control JTAG instruction encoding



Memory system

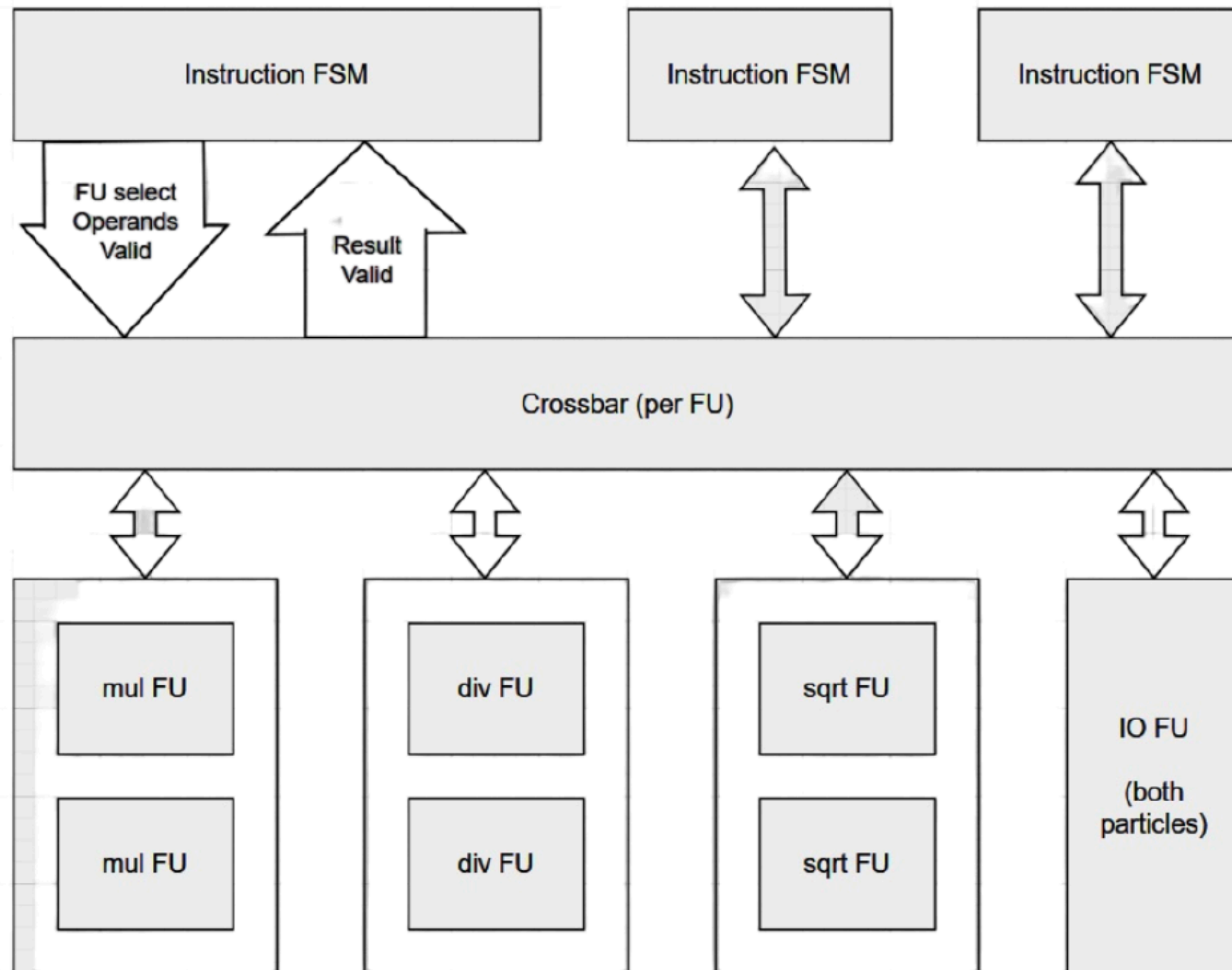
- Initial and subsequent evolved particle states are stored in 2 4096 line SRAM blocks, allowing for the simulation to handle 4096 particles
- Each block is able to be accessed and written to independently via JTAG, which happens when setting the initial conditions of the simulation
- To the rest of the system, they appear as one unified memory block. All the parameters of a particle are effectively on one line, optimising memory bandwidth (only need one read to get a particles information)



SRAM Line layout		
Instruction	Binary	Description
IR_NOP	0000	No operation
IR_READ	0011	Read from memory through JTAG
IR_DONE	0100	Close JTAG interface and release CPU reset
IR_WRITE	0101	Write to memory through JTAG
IR_MEM_CTRL	0111	Isolate memory from CPU for safe programming

SRAM JTAG instruction encoding

Velocity Updator – Main Processor



VU calculates acceleration of a particle:

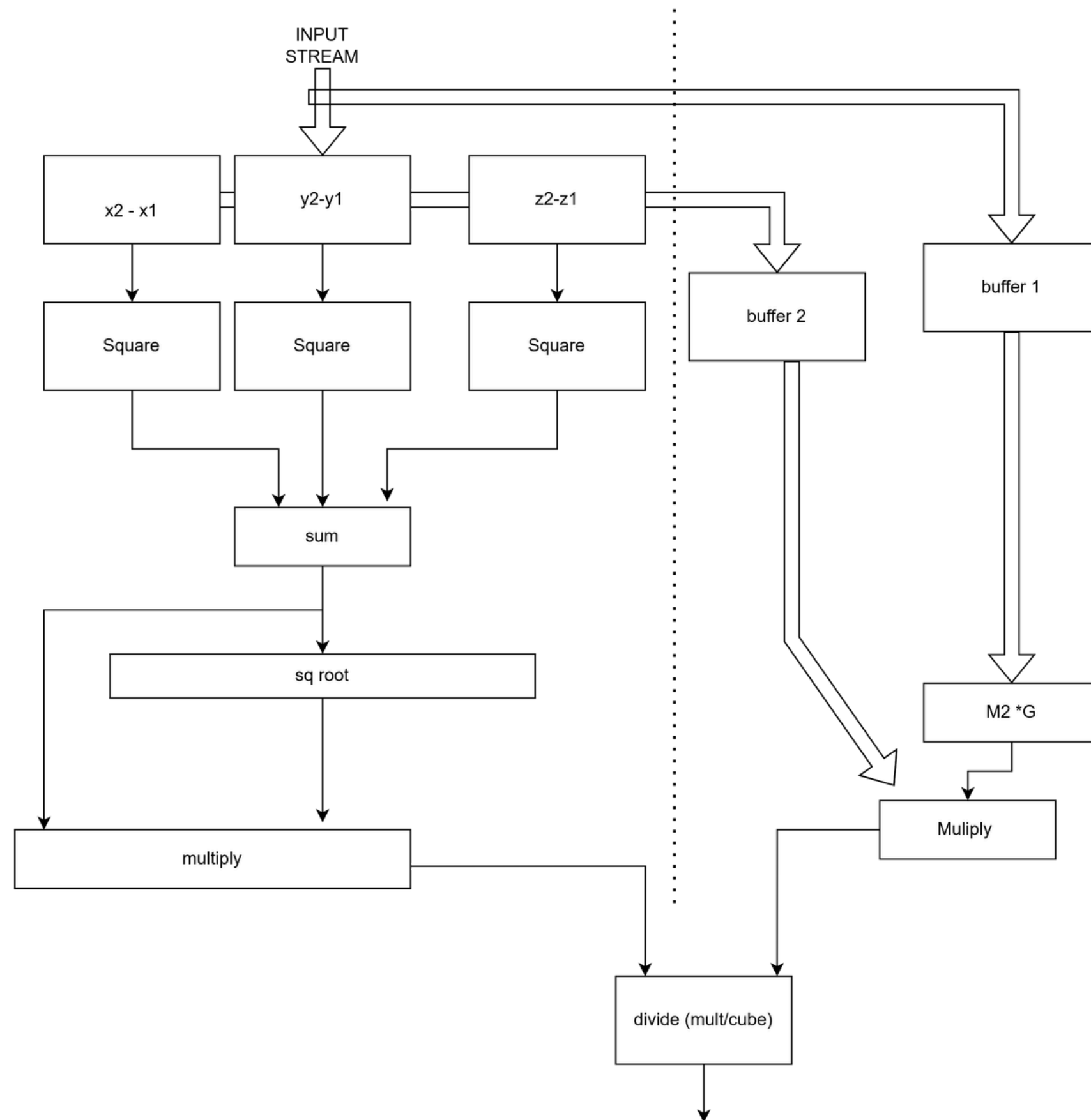
Pairwise Force Calculation

Stream other particles
(Load a particle + Stream ...
Stream ...)

Returned result applied as net acceleration.

1. SoC performance critical component
2. Multiple Instantiation could support parallelisation

Initial Velocity Approach: unrolled pipeline

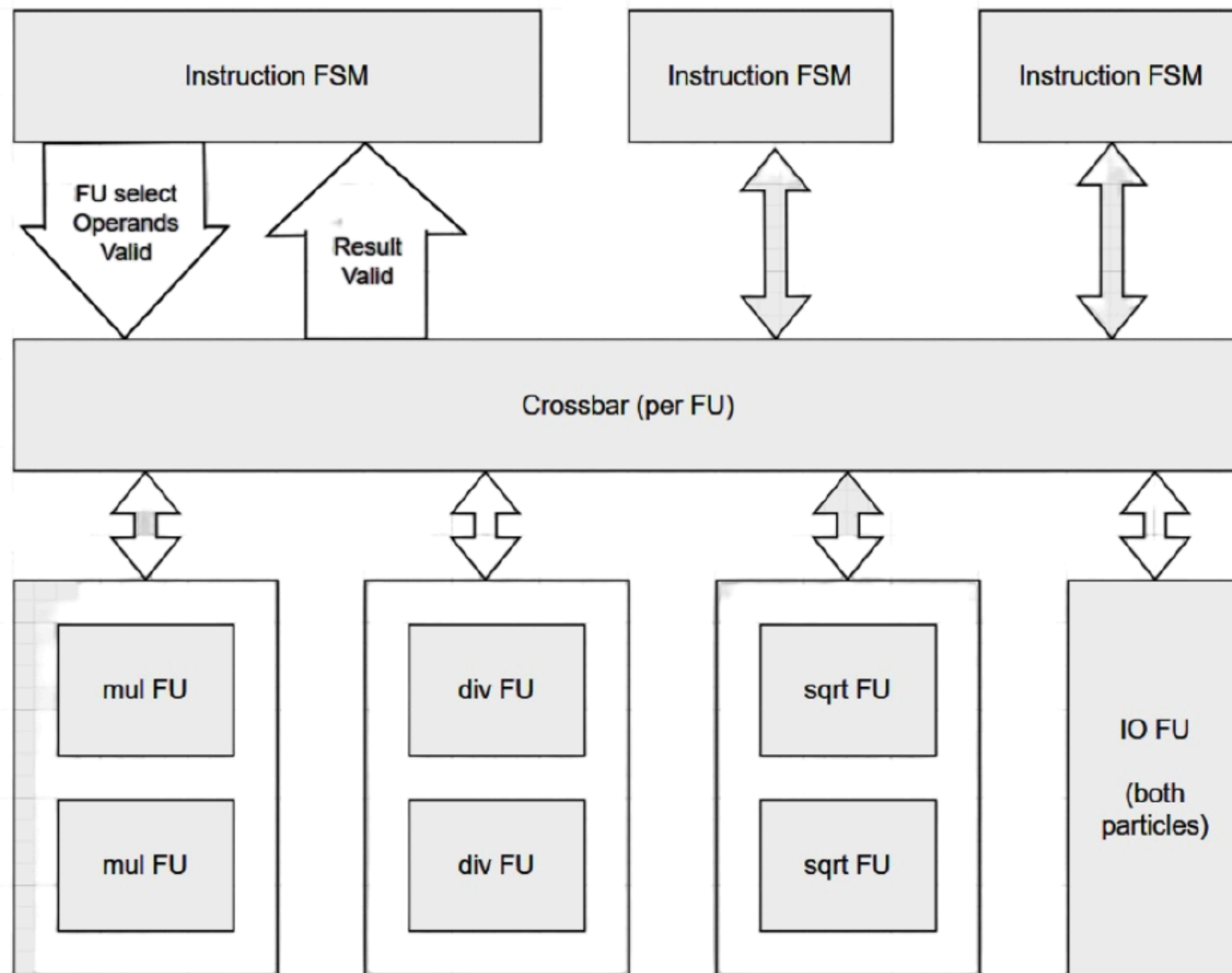


Aim : Unroll everything for maximum throughput

Cons: Massive area limitations

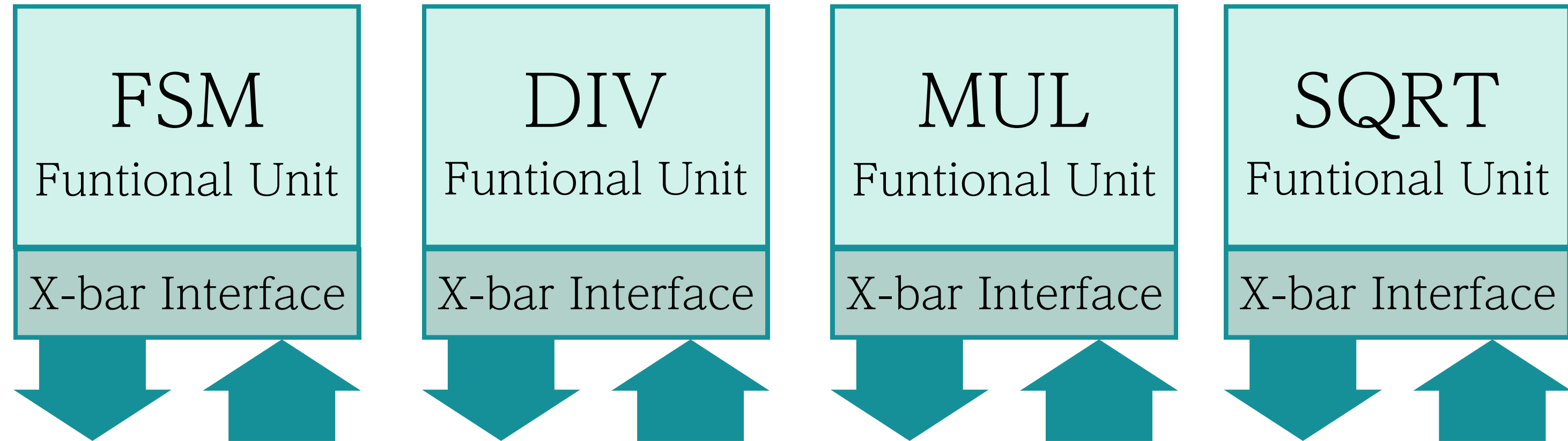
- To preserve accuracy with fixed point arithmetic, each successive stage takes in and uses longer bit-widths
- With our chosen parameters this meant the divisor module operated on bit widths of ~ 60 , which fully unrolled required 136 stages
- 1 such velocity updater took up nearly 4x our available die-space and thus required an almost complete redesign of the VU

Refine Velocity Approach: Shared Functional Unit



- Reduce silicon area with tradeoff of increasing total cycles
- Using FSMs to control the state of a calculation
- Cross bar becomes the important component to manage task scheduling

FSMs and FUs

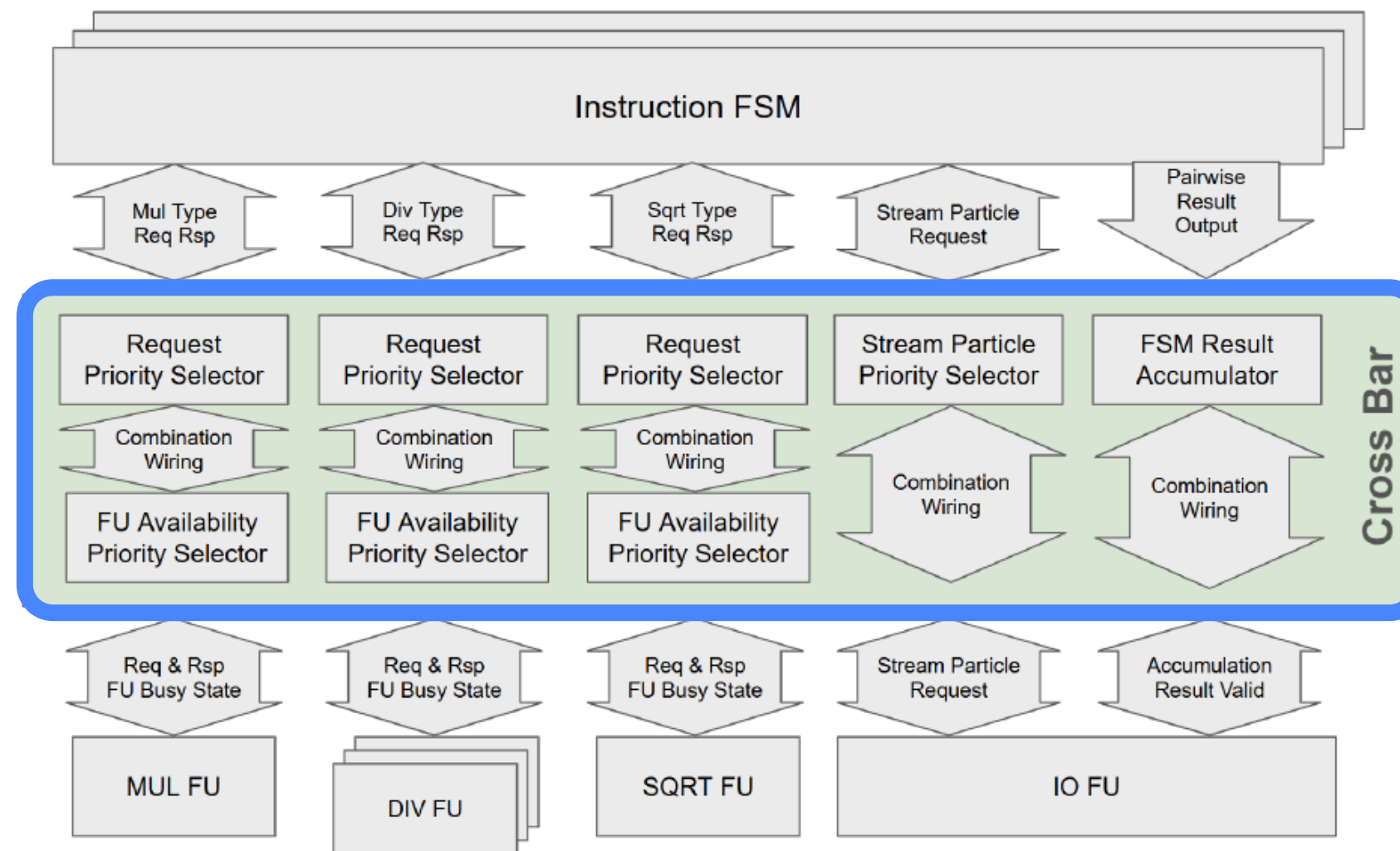


- VU core contains mutiple FSMs & FUs
- FUs reused at different stage of equation

FSMs $\longleftrightarrow$ Cross Bar $\longleftrightarrow$ FUs

- Performance are scalable with silicon area for more FSMs & FUs

Cross bar



Cross Bar

Features

- Cross bar handle multiple types of FSM $\rightleftharpoons$ FU transaction exchange in parallel

- Auto schedule FSM task for clashed FU requests.
- Adapts to any number of FSM request to keep pipeline full.

- Auto-distribute task to all FUs.
- Scalable for more FUs, as it is agnostic to FU count.

Cross bar

Challenge1:

Multi-request types,
multi-response types

Have to route request from:
FSM=>different FU types
Have to route response from:
different FU types =>FSM

Track each FU type process status
with individual status queue

Proposed Solution

Result



- Cross bar handle multiple types of FSM $\rightleftharpoons$ FU transaction exchange in parallel

Challenge2:

Scheduling task to functional unit

Have to manage FSM request to
keep functional unit busy

Leftmost Priority Encoder for FSM
request+ FSM-FU handshake

Proposed Solution

Result



- Auto schedule FSM task for clashed FU requests.
- Adapts to any number of FSM request to keep pipeline full.

Challenge3:

Load balancing on functional units

Have to identify idle FU and assign
task from FSM

Leftmost Priority Encoder
for FU idle index

Proposed Solution

Result



- Auto-distribute task to all FUs.
- Scalable for more FUs, as it is agnostic to FU count.

Cross bar

1

Challenge1:
Multi-request types,
multi-response types

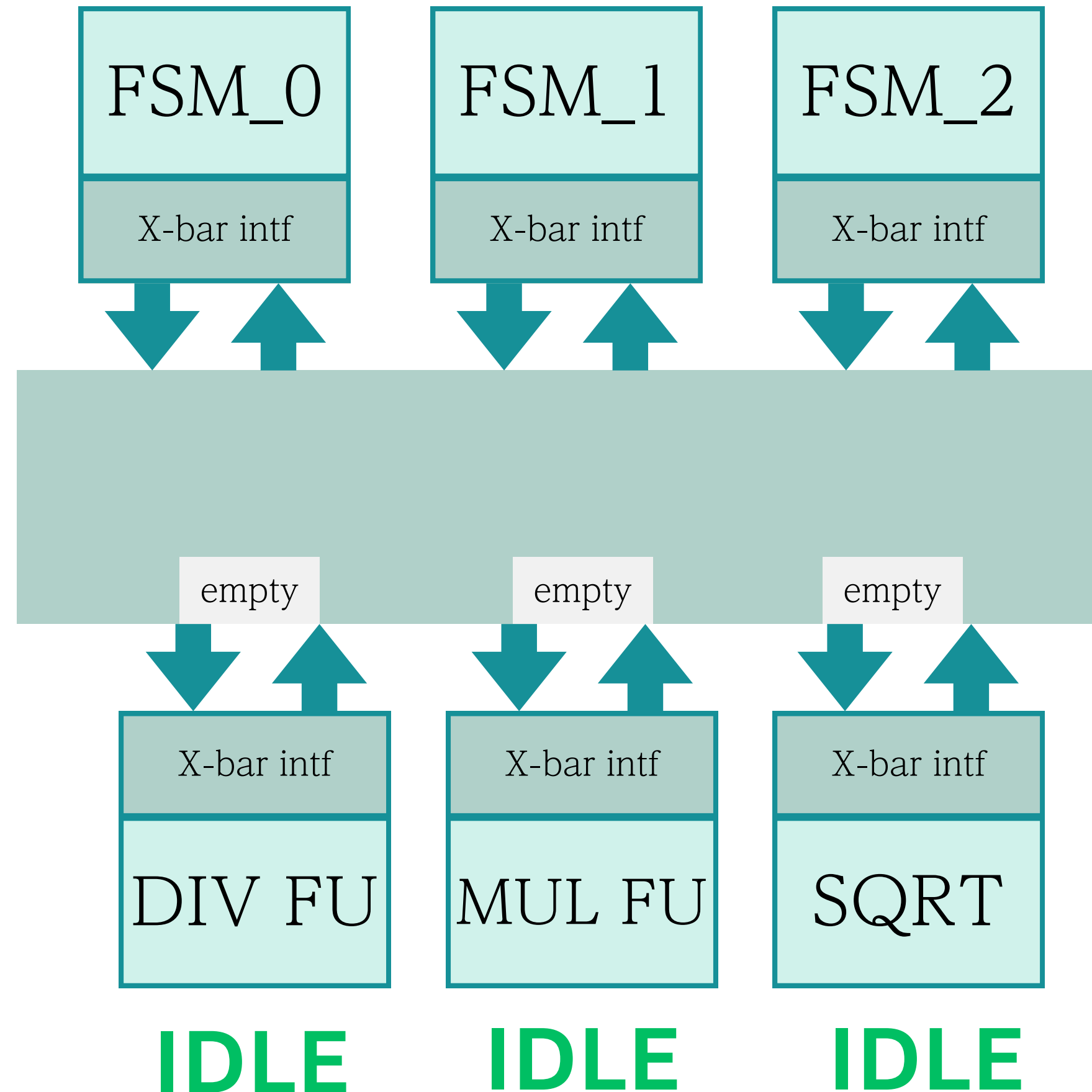
Have to route request from:
FSM=>different FU types
Have to route response from:
different FU types =>FSM

Track each FU type process status
with individual status queue

Proposed Solution

Result

- Cross bar handle multiple types of FSM $\leftrightarrow$ FU transaction exchange in parallel



Cross bar

1

Challenge1:
Multi-request types,
multi-response types

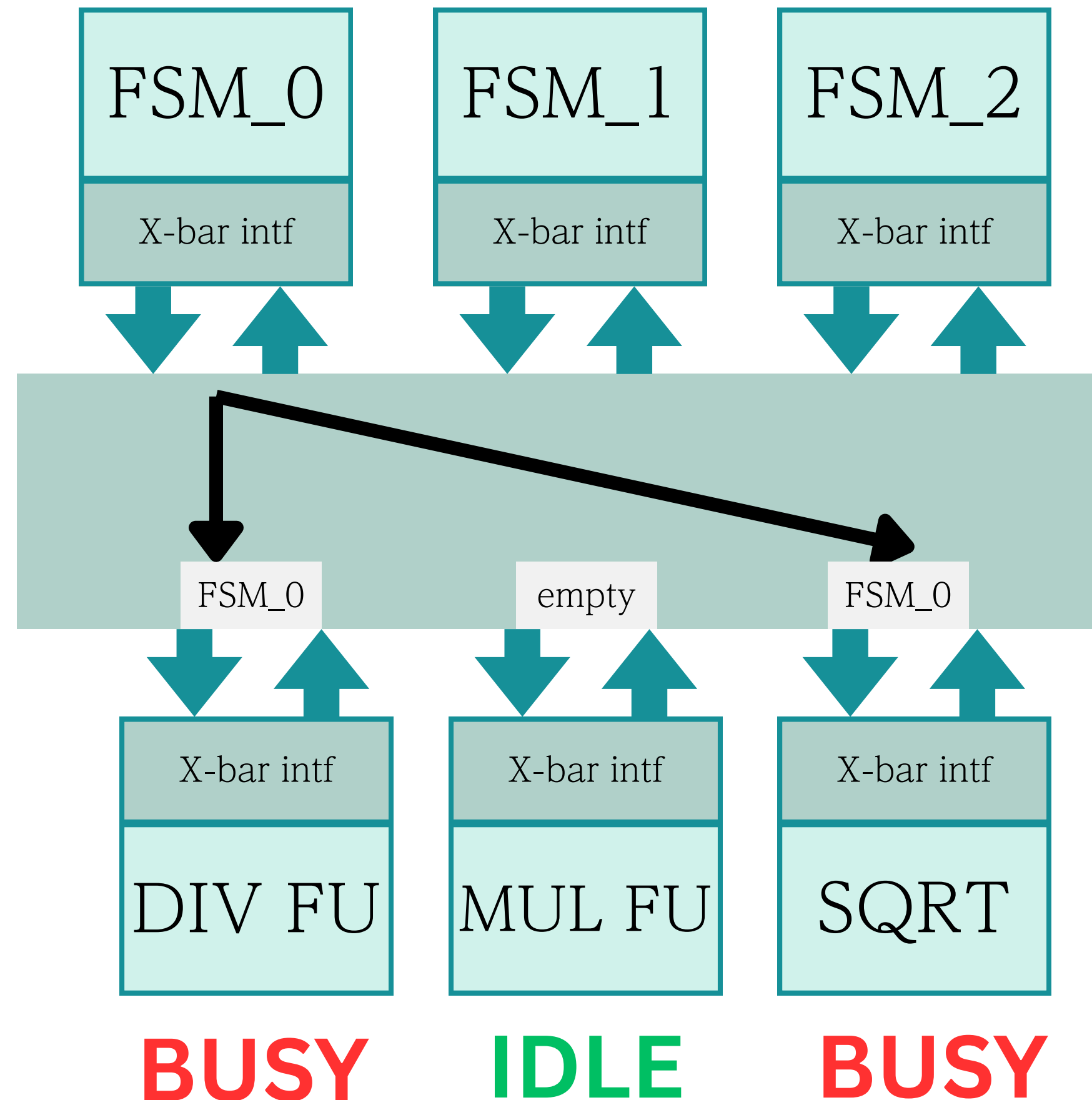
Have to route request from:
FSM=>different FU types
Have to route response from:
different FU types =>FSM

Track each FU type process status
with individual status queue

Proposed Solution

Result

- Cross bar handle multiple types of FSM $\leftrightarrow$ FU transaction exchange in parallel



Cross bar

1

Challenge1:
Multi-request types,
multi-response types

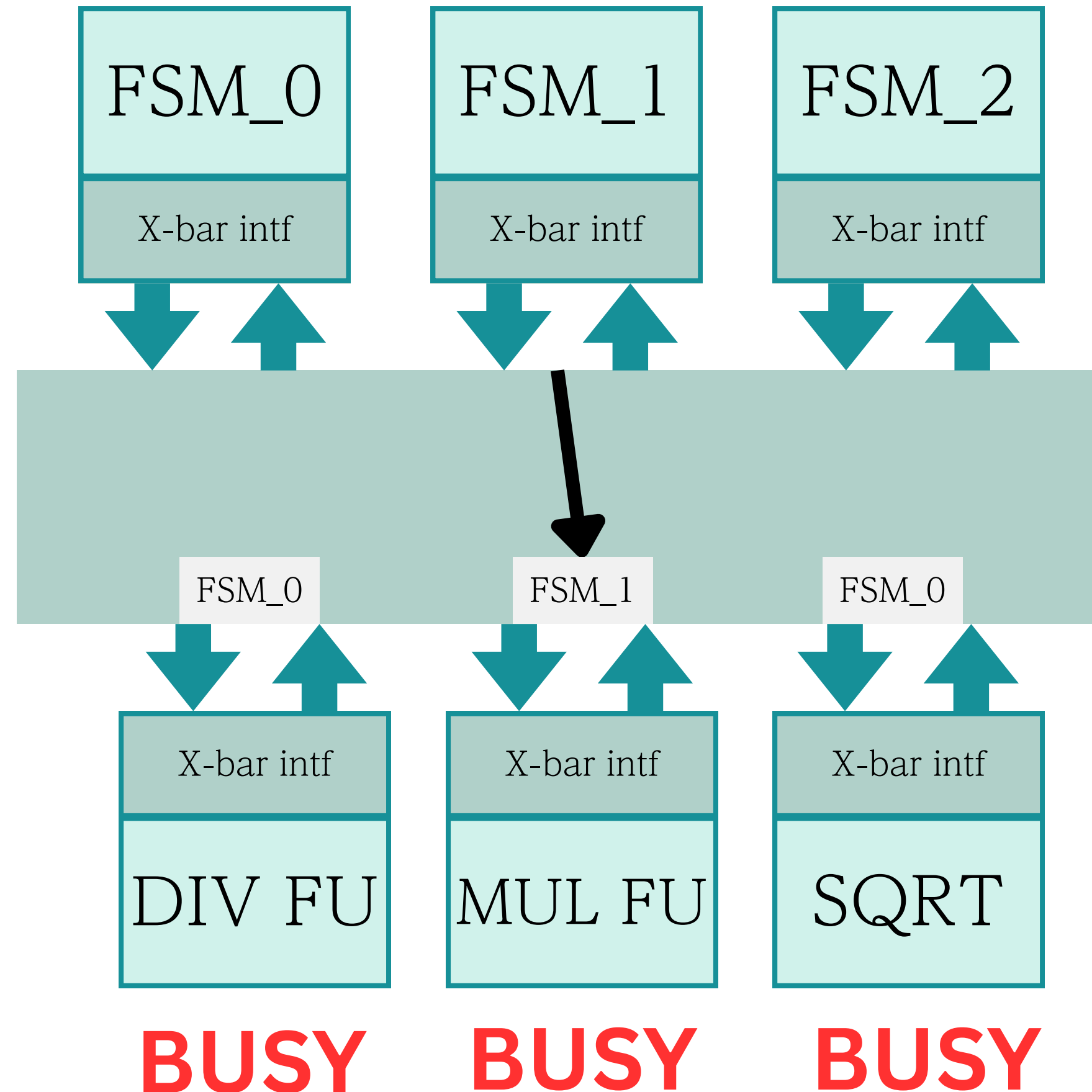
Have to route request from:
FSM=>different FU types
Have to route response from:
different FU types =>FSM

Track each FU type process status
with individual status queue

Proposed Solution

Result

- Cross bar handle multiple types of FSM $\leftrightarrow$ FU transaction exchange in parallel



Cross bar

1

Challenge1:
Multi-request types,
multi-response types

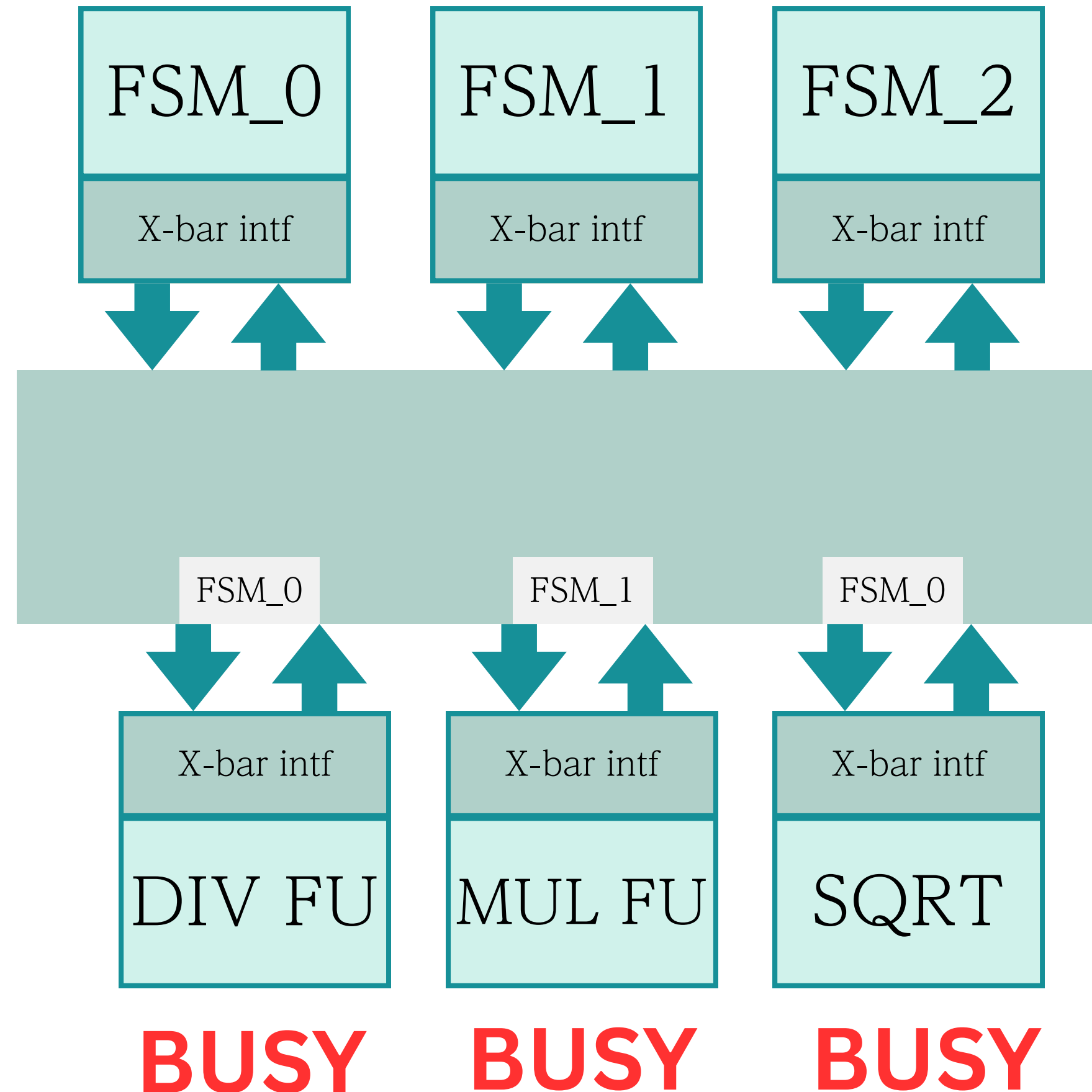
Have to route request from:
FSM=>different FU types
Have to route response from:
different FU types =>FSM

Track each FU type process status
with individual status queue

Proposed Solution

Result

- Cross bar handle multiple types of FSM $\leftrightarrow$ FU transaction exchange in parallel



Cross bar

1

Challenge1:
Multi-request types,
multi-response types

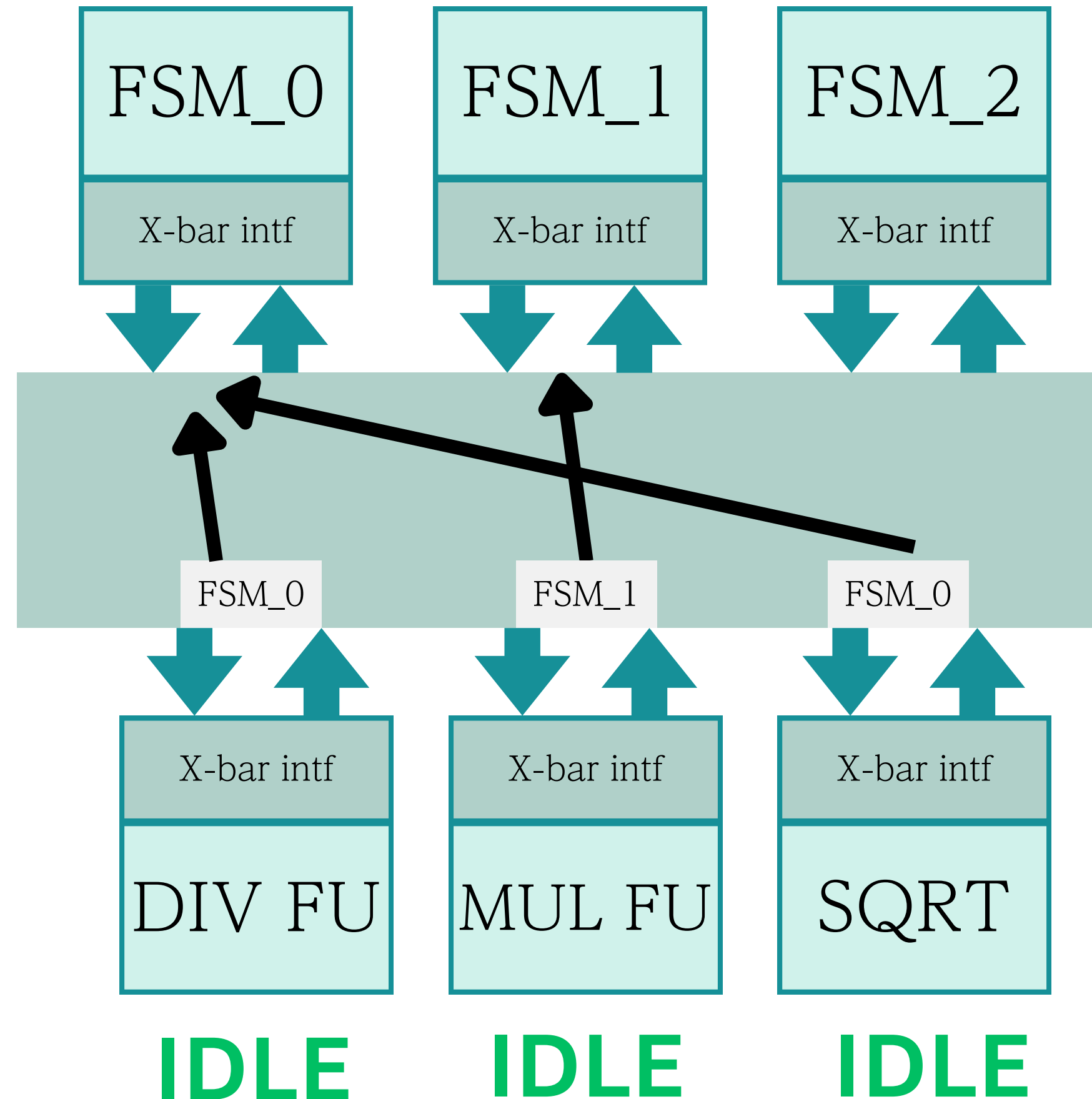
Have to route request from:
FSM=>different FU types
Have to route response from:
different FU types =>FSM

Track each FU type process status
with individual status queue

Proposed Solution

Result

- Cross bar handle multiple types of FSM $\rightleftharpoons$ FU transaction exchange in parallel



Cross bar

1

Challenge1:
Multi-request types,
multi-response types

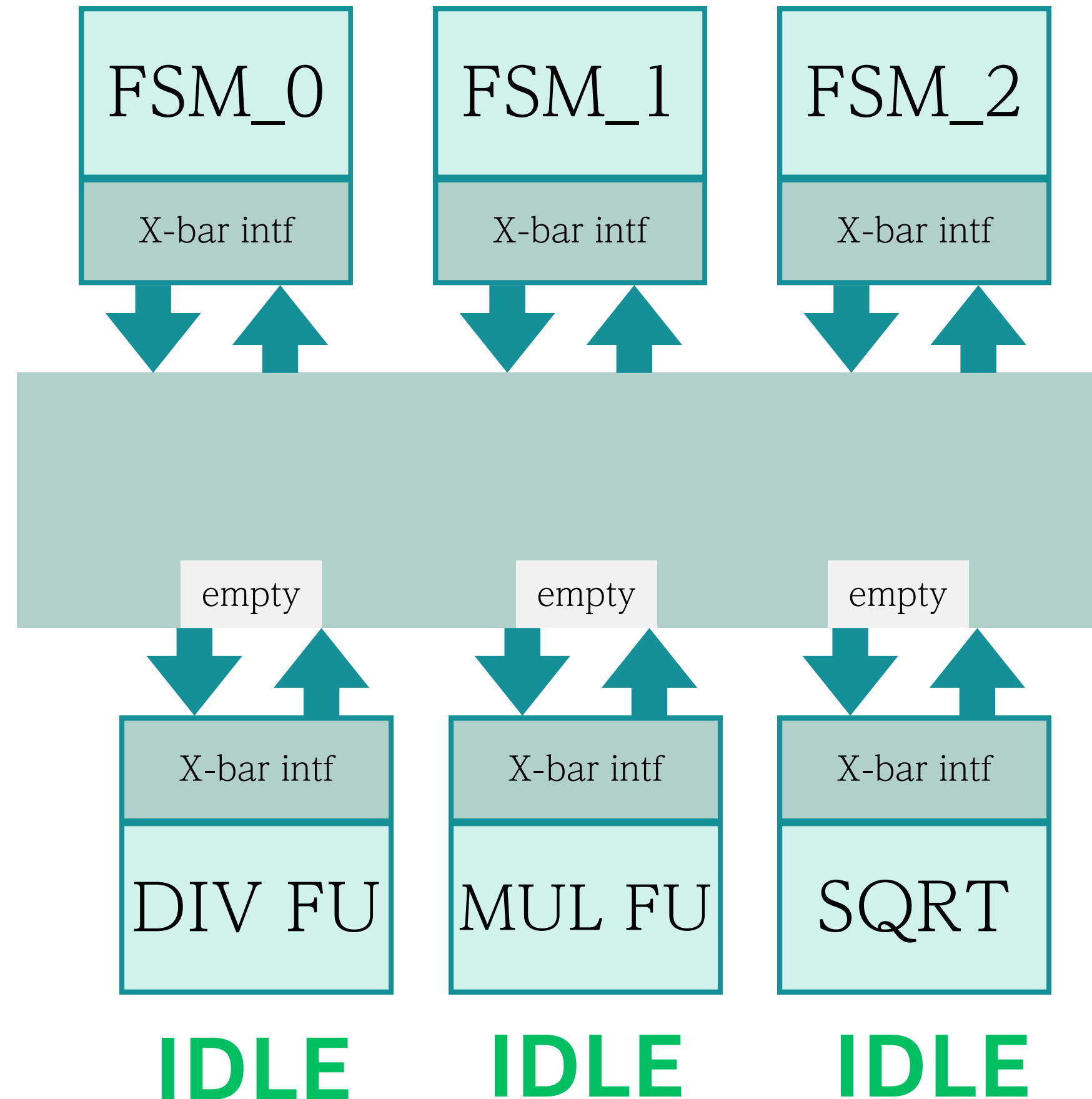
Have to route request from:
FSM=>different FU types
Have to route response from:
different FU types =>FSM

Track each FU type process status
with individual status queue

Proposed Solution

Result

- Cross bar handle multiple types of FSM $\leftrightarrow$ FU transaction exchange in parallel



Cross bar

2

Challenge2:

Scheduling task to functional unit

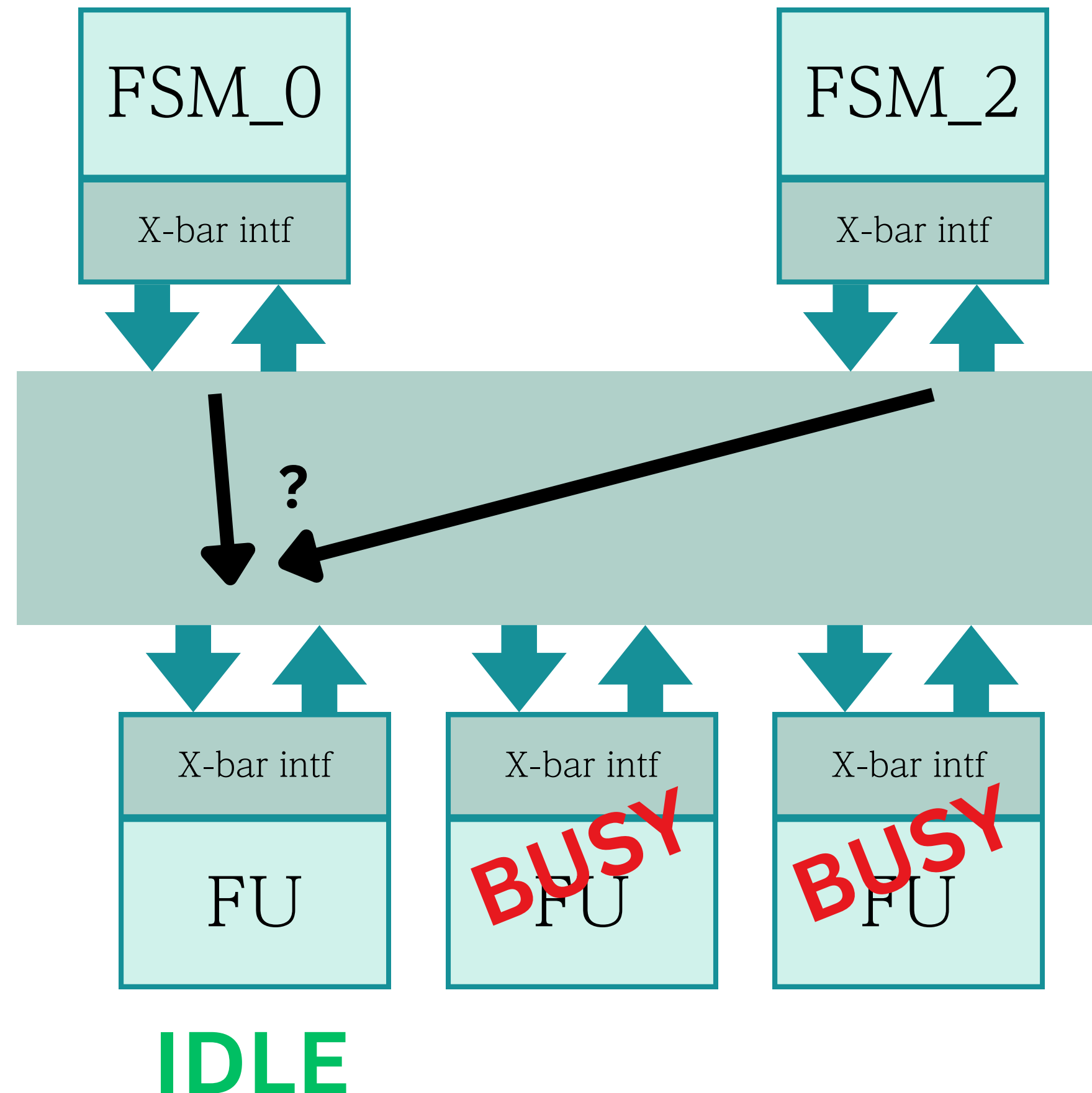
Have to manage FSM request to keep functional unit busy

Leftmost Priority Encoder for FSM request+ FSM-FU handshake

Proposed Solution

Result

- Auto schedule FSM task for clashed FU requests.
- Adapts to any number of FSM request to keep pipeline full.



Cross bar

2

Challenge2:

Scheduling task to functional unit

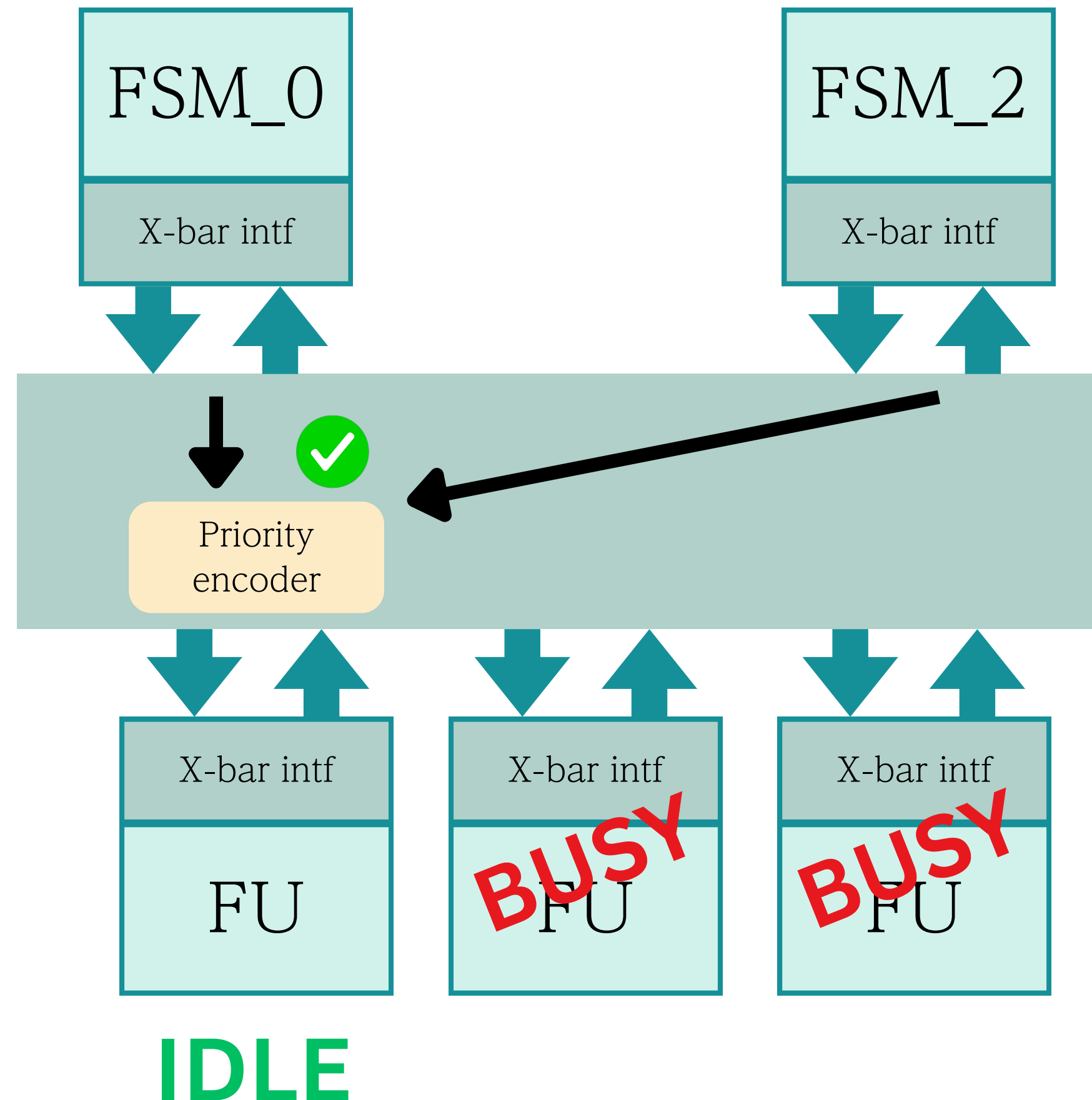
Have to manage FSM request to keep functional unit busy

Leftmost Priority Encoder for FSM request+ FSM-FU handshake

Proposed Solution

Result

- Auto schedule FSM task for clashed FU requests.
- Adapts to any number of FSM request to keep pipeline full.



Cross bar

3

Challenge3:

Load balancing on functional units

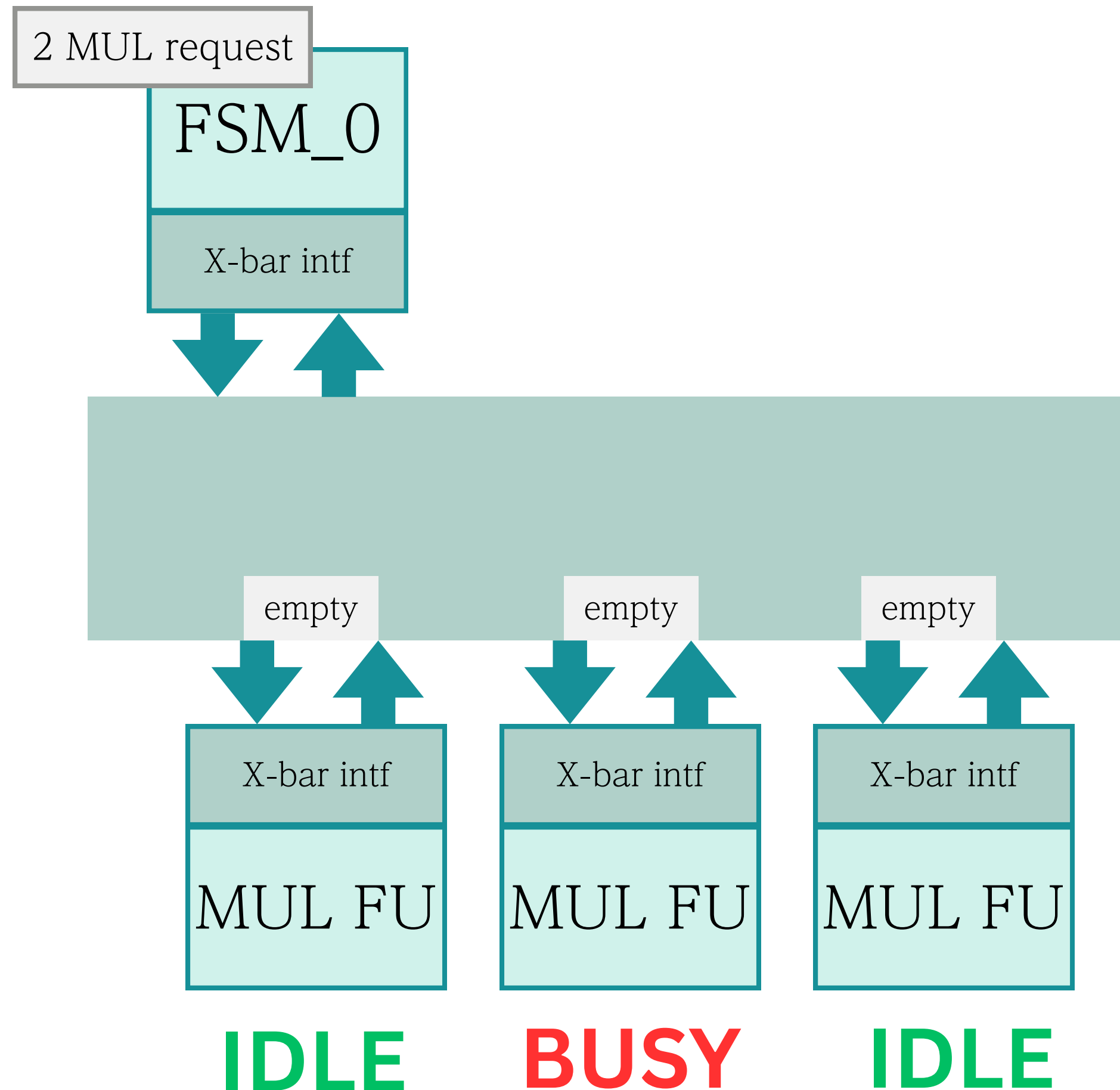
Have to identify idle FU and assign task from FSM

Leftmost Priority Encoder
for FU idle index

Proposed Solution

Result

- Auto-distribute task to all FUs.
- Scalable for more FUs, as it is agnostic to FU count.



Cross bar

3

Challenge3:

Load balancing on functional units

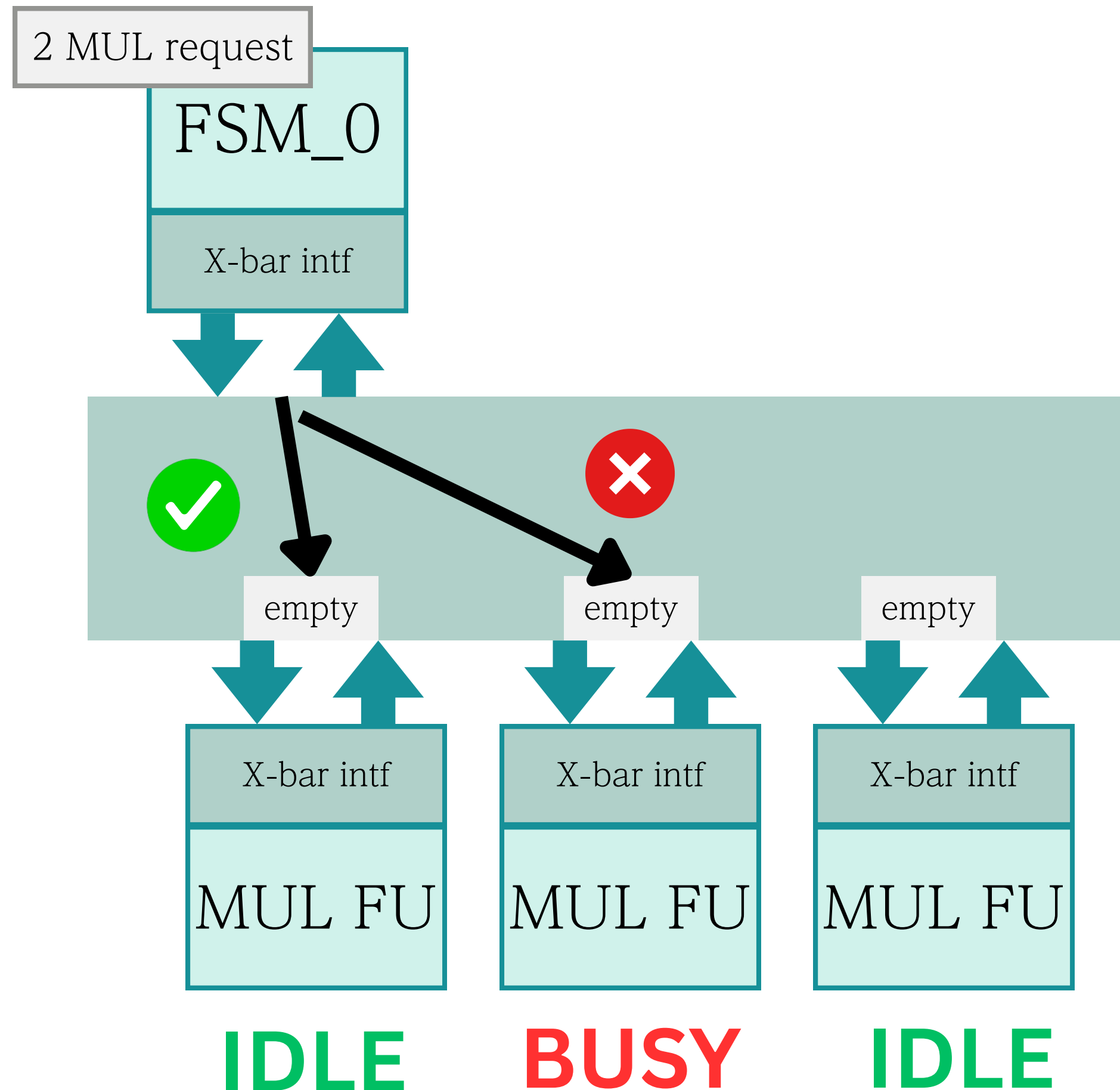
Have to identify idle FU and assign task from FSM

Leftmost Priority Encoder
for FU idle index

Proposed Solution

Result

- Auto-distribute task to all FUs.
- Scalable for more FUs, as it is agnostic to FU count.



Cross bar

3

Challenge3:

Load balancing on functional units

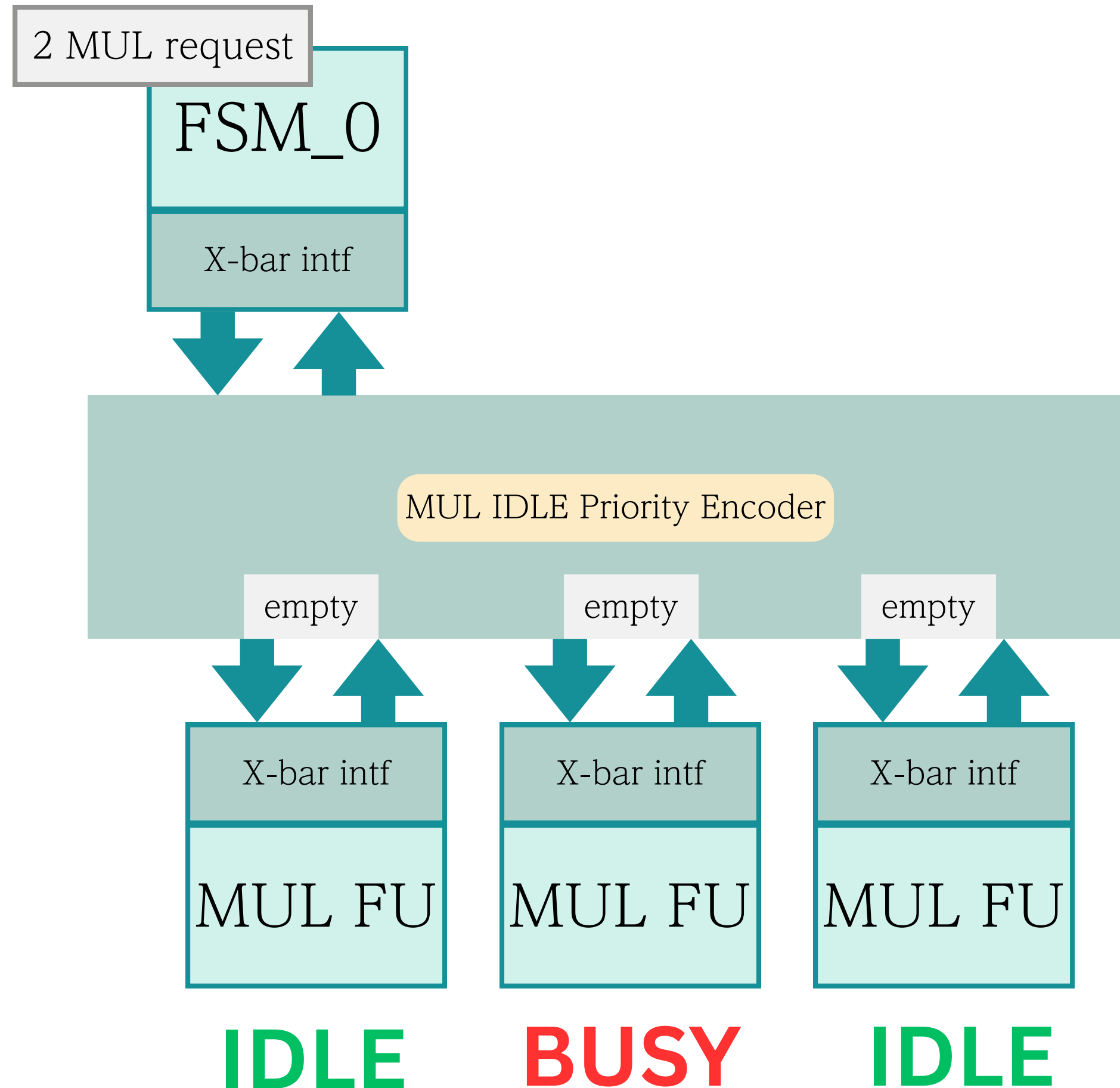
Have to identify idle FU and assign task from FSM

Leftmost Priority Encoder
for FU idle index

Proposed Solution

Result

- Auto-distribute task to all FUs.
- Scalable for more FUs, as it is agnostic to FU count.



Cross bar

3

Challenge3:

Load balancing on functional units

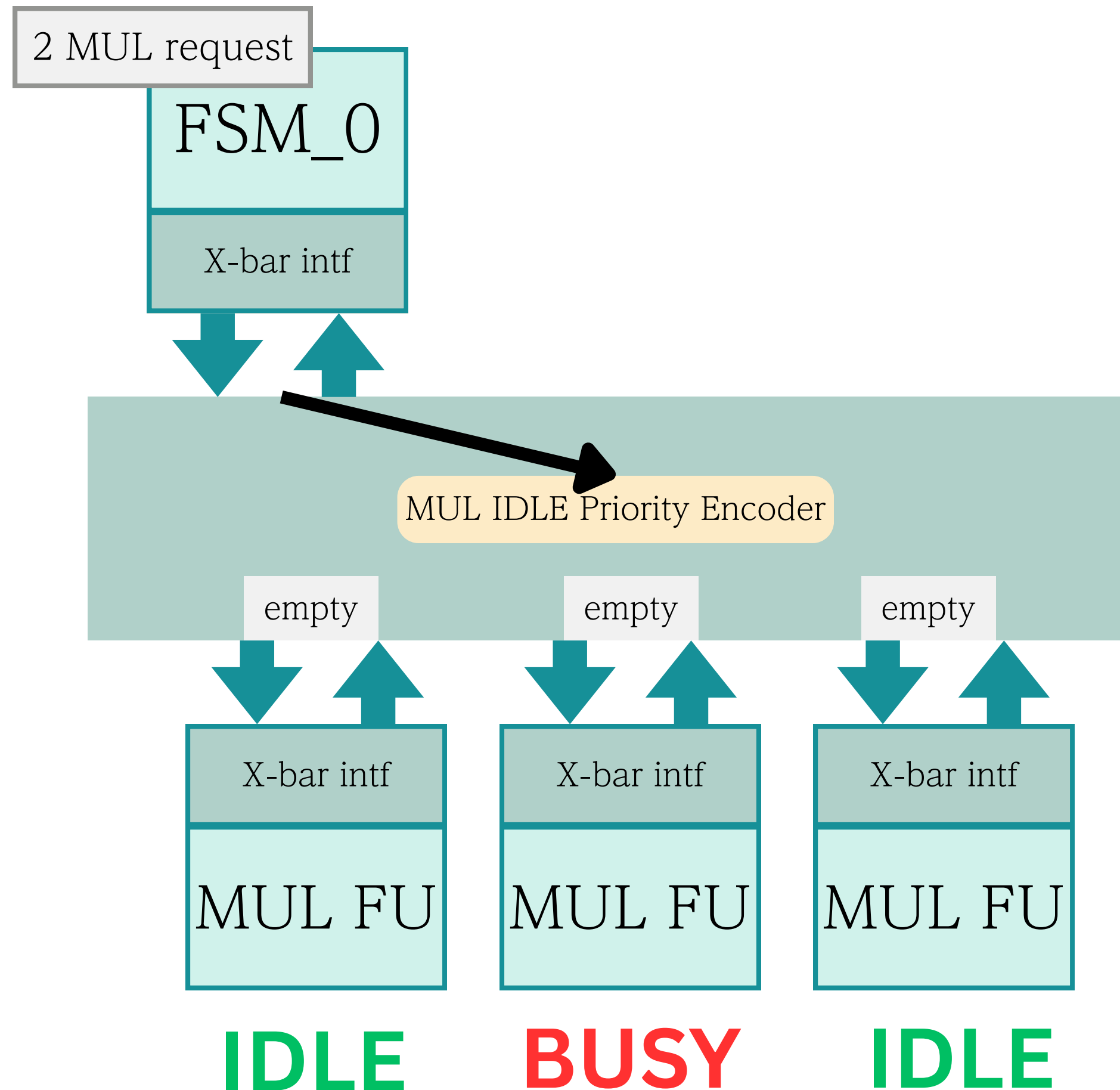
Have to identify idle FU and assign task from FSM

Leftmost Priority Encoder
for FU idle index

Proposed Solution

Result

- Auto-distribute task to all FUs.
- Scalable for more FUs, as it is agnostic to FU count.



Cross bar

3

Challenge3:

Load balancing on functional units

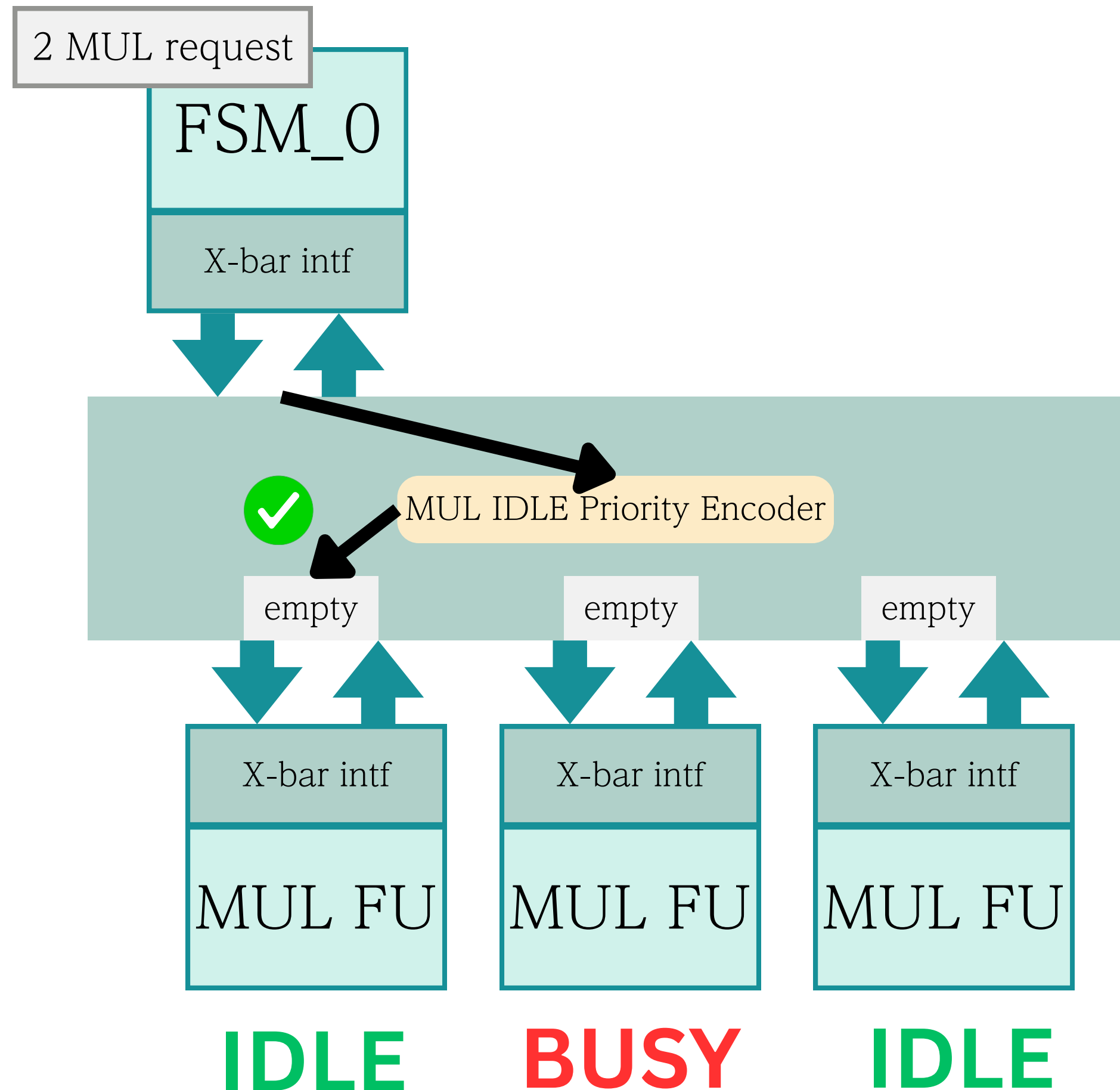
Have to identify idle FU and assign task from FSM

Leftmost Priority Encoder
for FU idle index

Proposed Solution

Result

- Auto-distribute task to all FUs.
- Scalable for more FUs, as it is agnostic to FU count.



Cross bar

3

Challenge3:

Load balancing on functional units

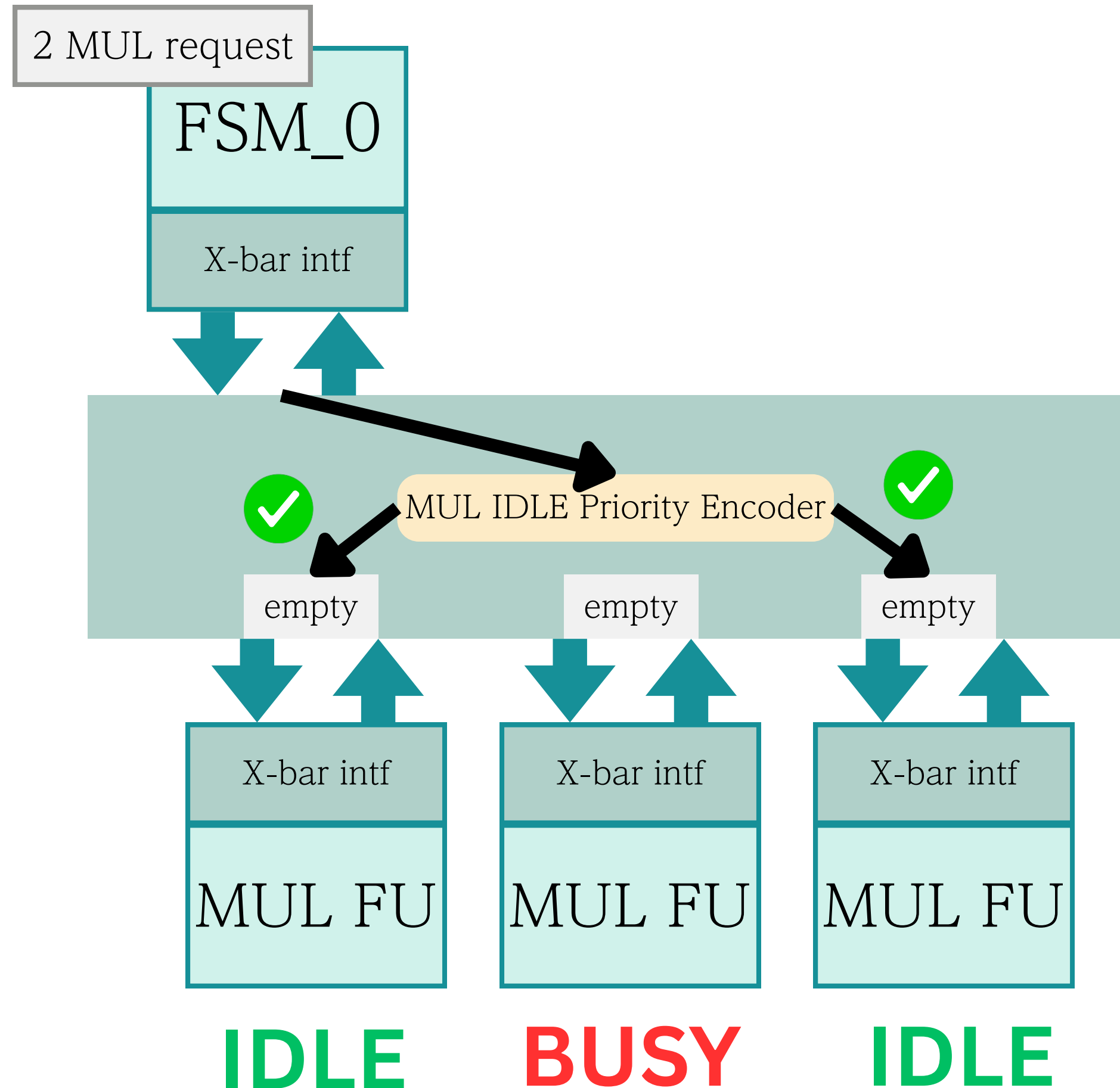
Have to identify idle FU and assign task from FSM

Leftmost Priority Encoder
for FU idle index

Proposed Solution

Result

- Auto-distribute task to all FUs.
- Scalable for more FUs, as it is agnostic to FU count.



Cross bar

Challenge1:

Multi-request types,
multi-response types

Have to route request from:
FSM=>different FU types
Have to route response from:
different FU types =>FSM

Track each FU type process status
with individual status queue

Proposed Solution

Result



- Cross bar handle multiple types of FSM $\leftrightarrow$ FU transaction exchange in parallel

Challenge2:

Scheduling task to functional unit

Have to manage FSM request to
keep functional unit busy

Leftmost Priority Encoder for FSM
request+ FSM-FU handshake

Proposed Solution

Result



- Auto schedule FSM task for clashed FU requests.
- Adapts to any number of FSM request to keep pipeline full.

Challenge3:

Load balancing on functional units

Have to identify idle FU and assign
task from FSM

Leftmost Priority Encoder
for FU idle index

Proposed Solution

Result



- Auto-distribute task to all FUs.
- Scalable for more FUs, as it is agnostic to FU count.

Cross bar: optimisation

Multiple FSMs shares multiple FUs

Utilisation % of each FU types can be observed in the wave signal.

- Identify bottleneck for silicon resources optimation
- Remove bottleneck of a particular FU resources
- Easy scale up optimisation

Parameter	Value
FSM_NUM	4
MUL_NUM	1
DIV_NUM	3
SQRT_NUM	1

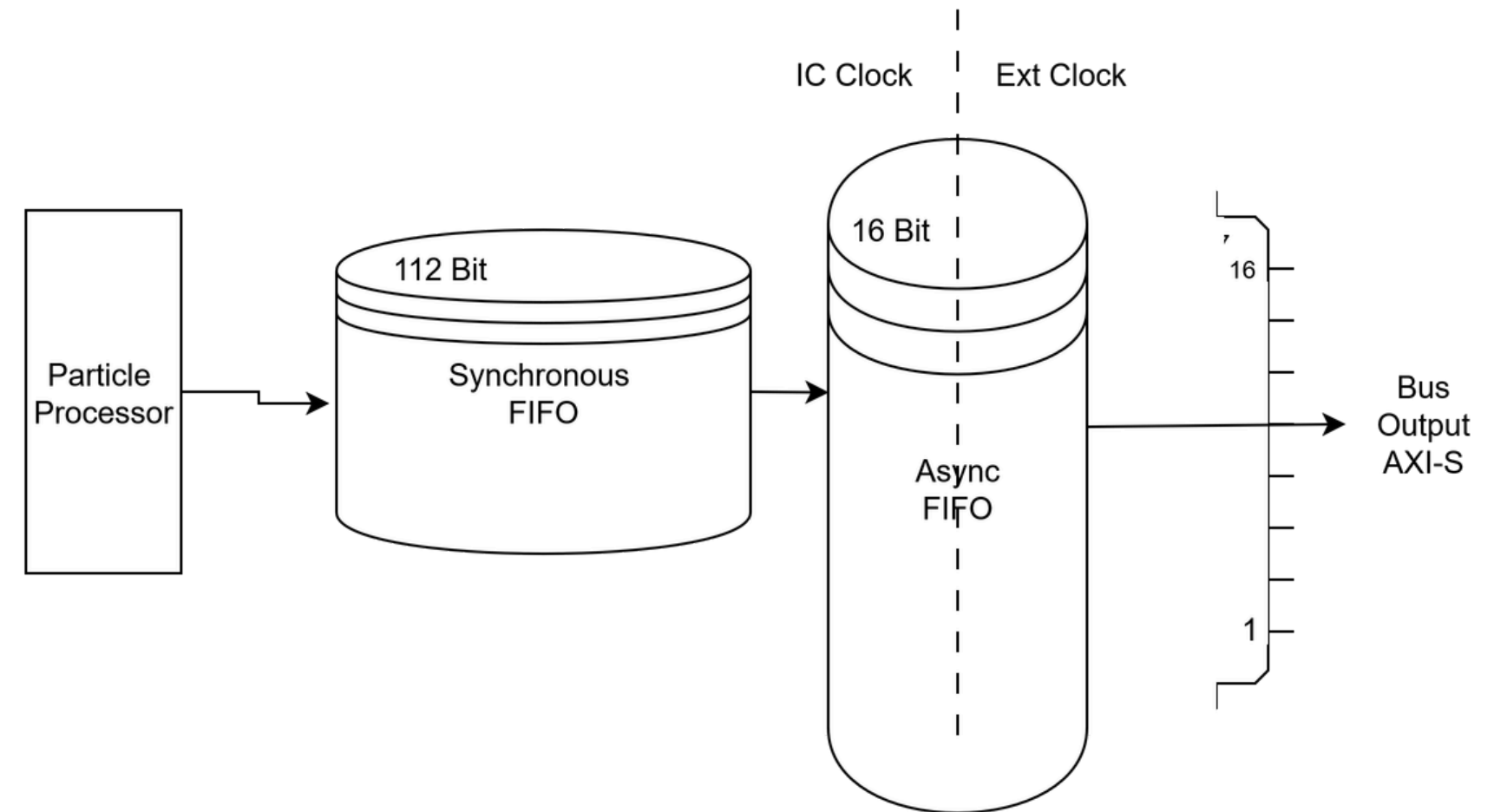
Found to be most
area/performance effective
configuration for accelerator



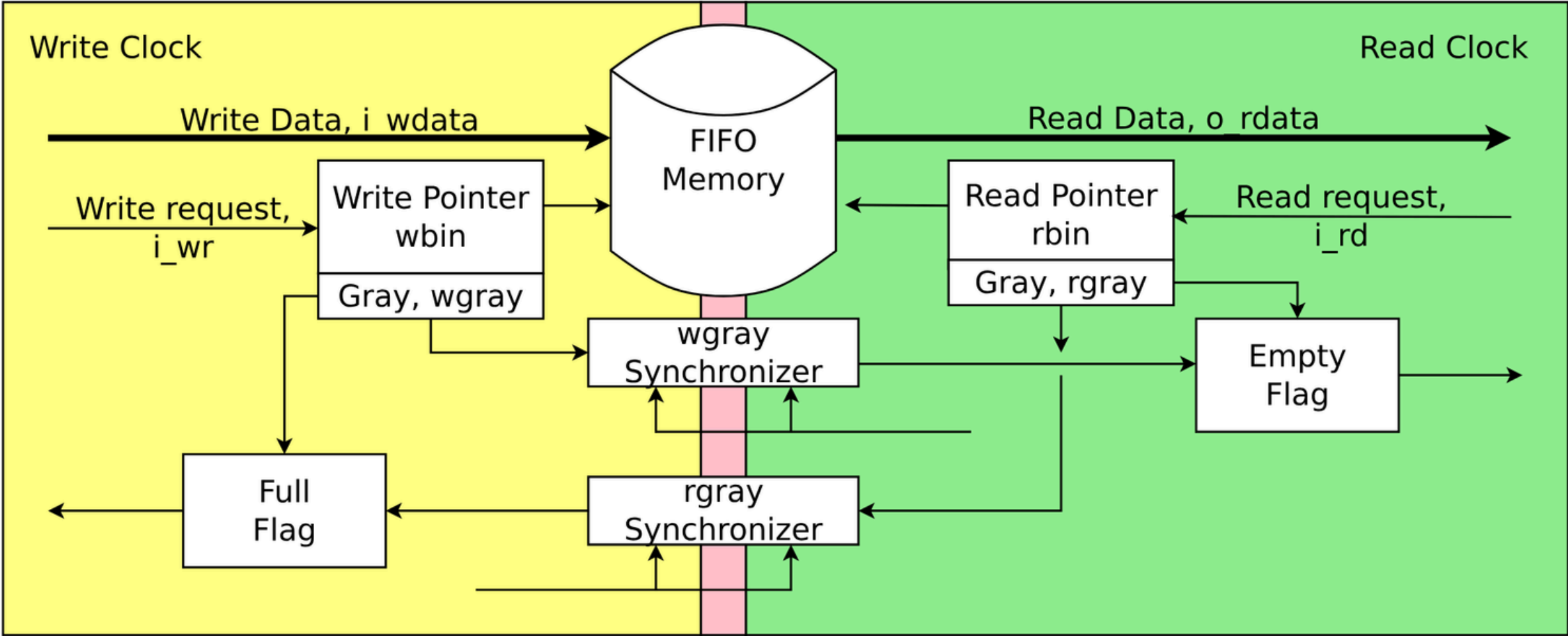
Resource Utilisation Signal

Output tap

- Allow for communication even with fluctuations in consumer and producer rates
- Allow for communication across the clock domains

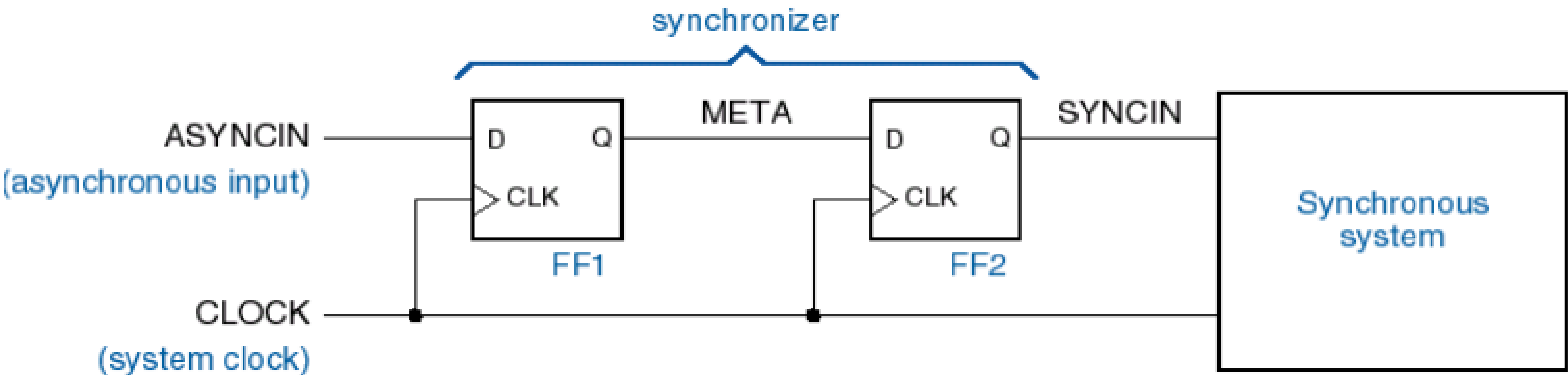


Async CDC FIFO



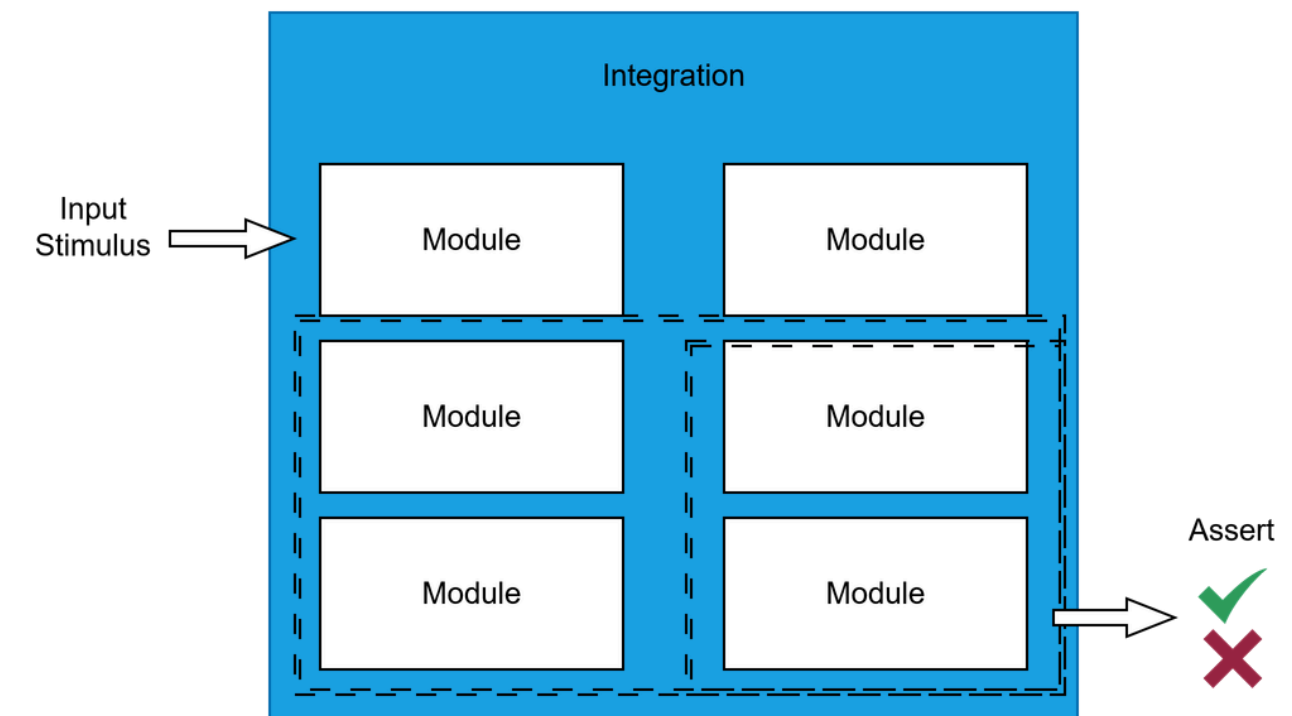
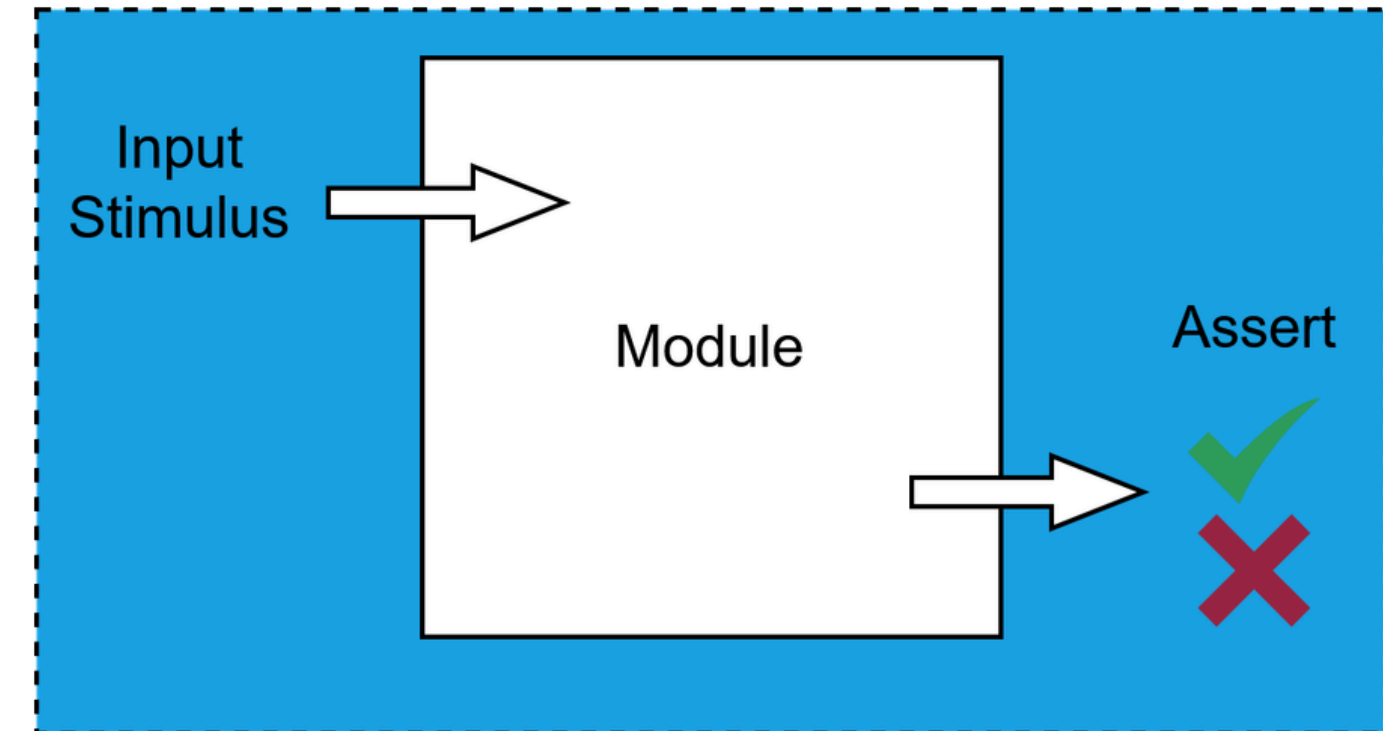
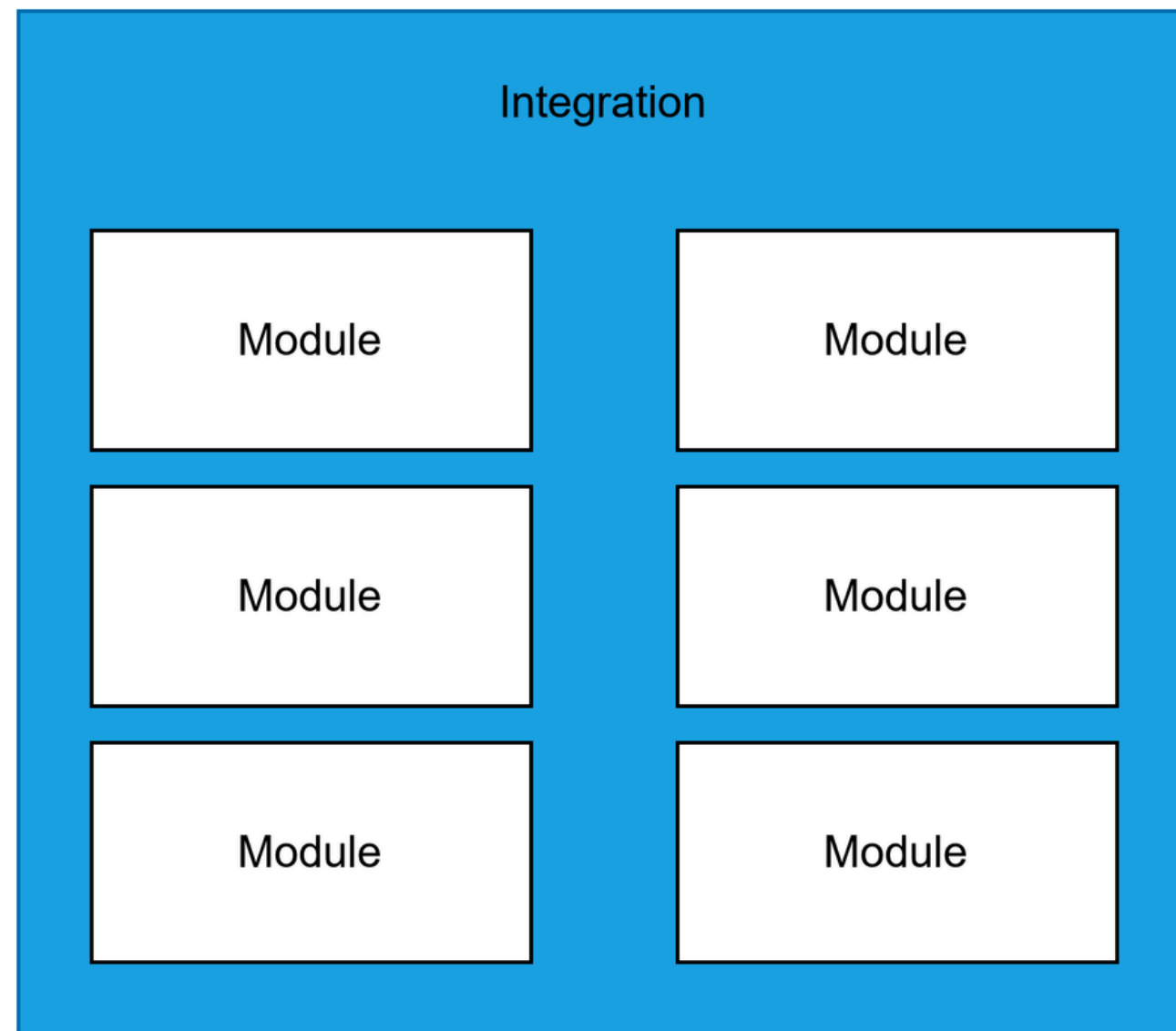
Counter	Gray Code
00000	00000
00001	00001
00010	00011
00011	00010
00100	00110
00101	00111
00110	00101
00111	00100
01000	01100
01001	01101
01010	01111
01011	01110
01100	01010
01101	01011
01110	01001
01111	01000
...	...

$$\text{gray_value} = (\text{counter} \gg 1) \wedge \text{counter};$$

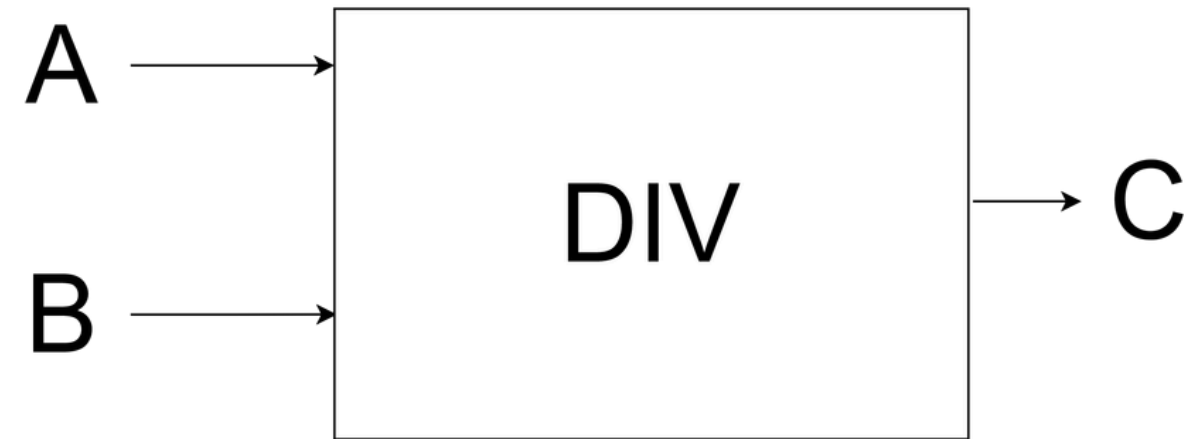


Verification

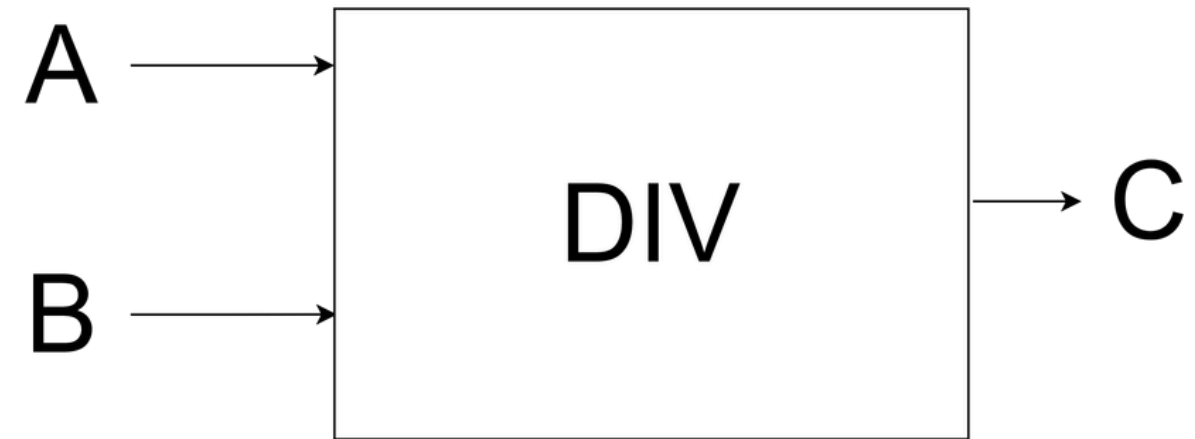
RTL Verification



Module Verification

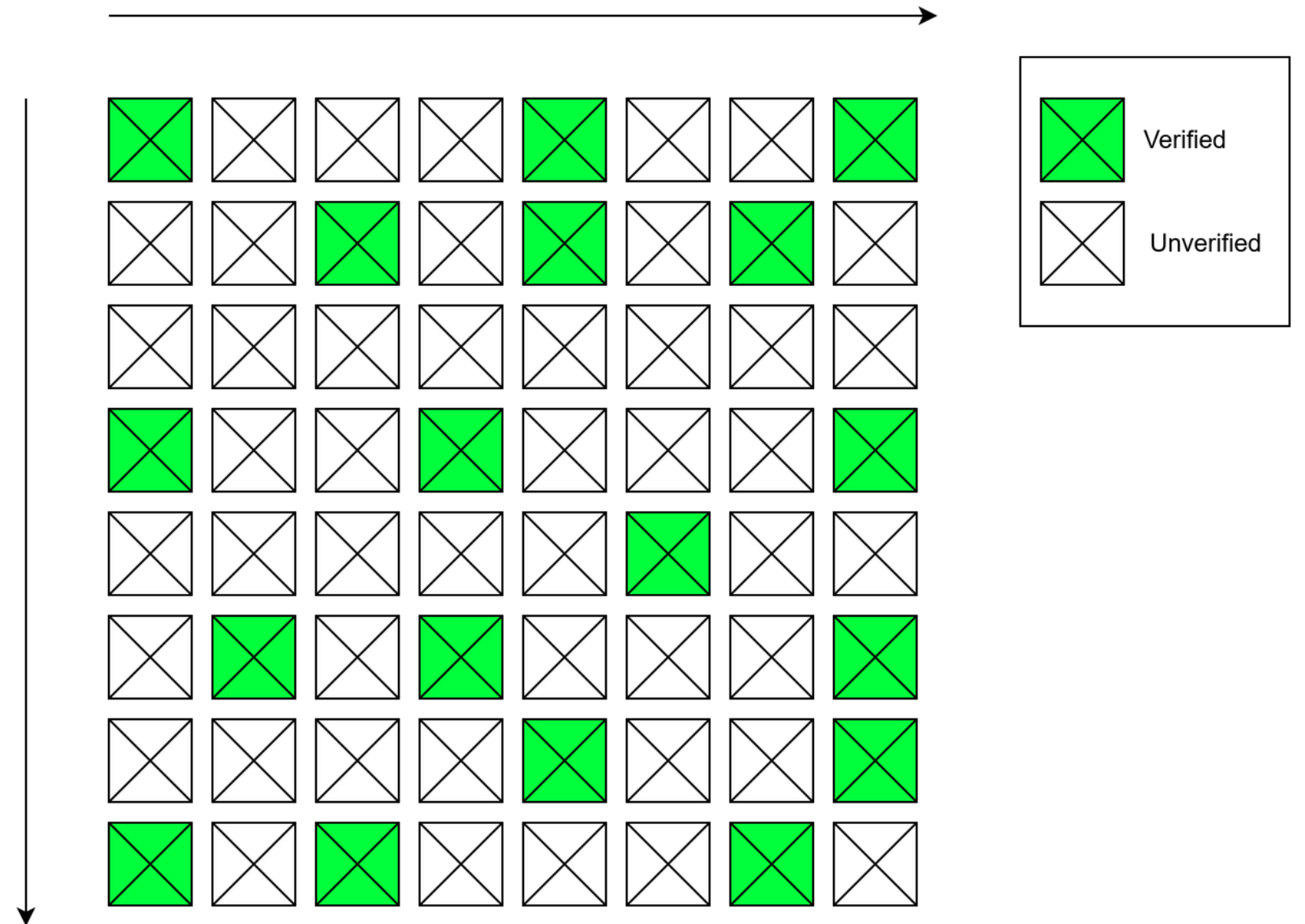
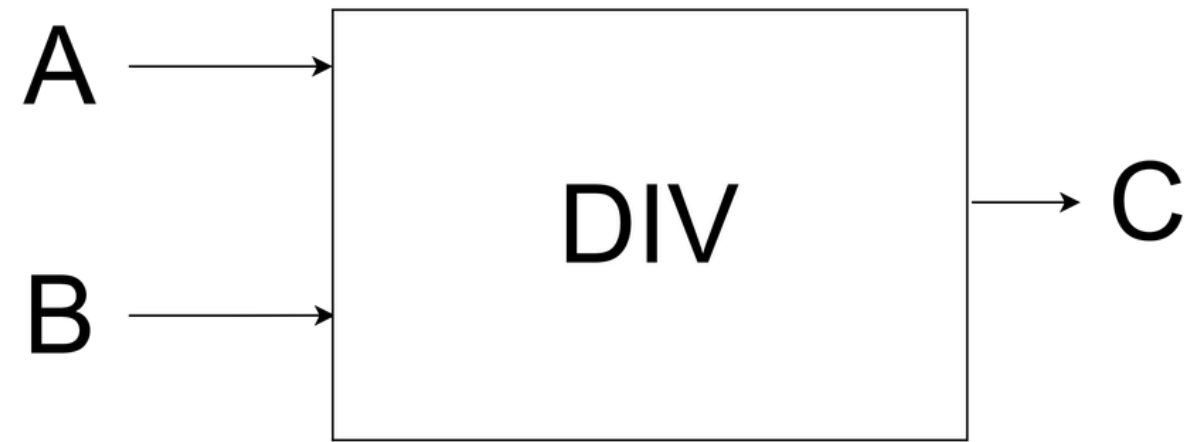


Module Verification

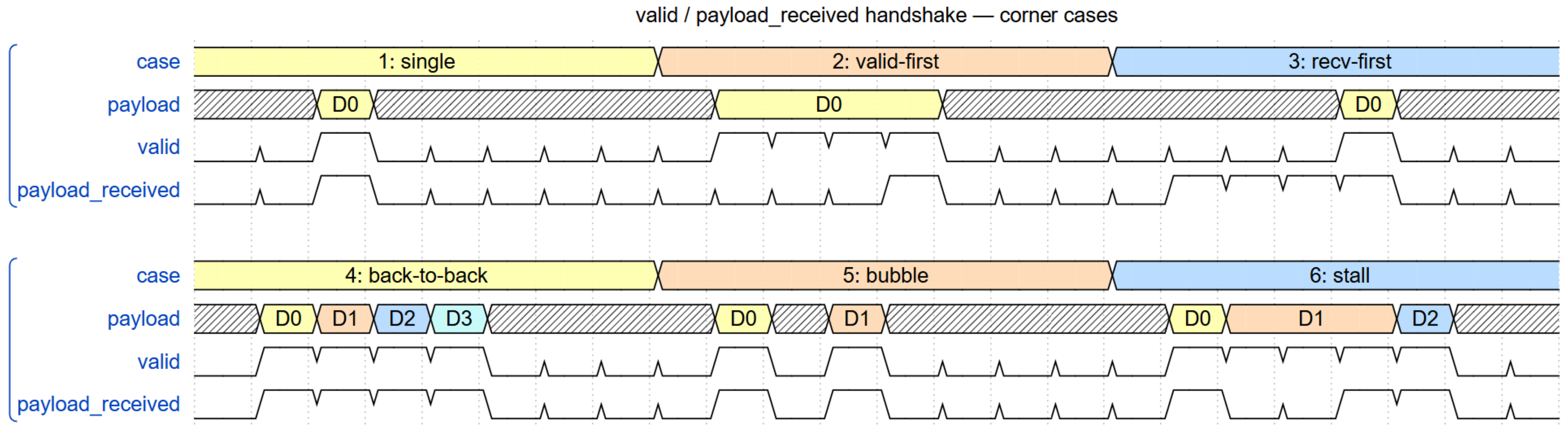


2.2300745e+43

Module Verification



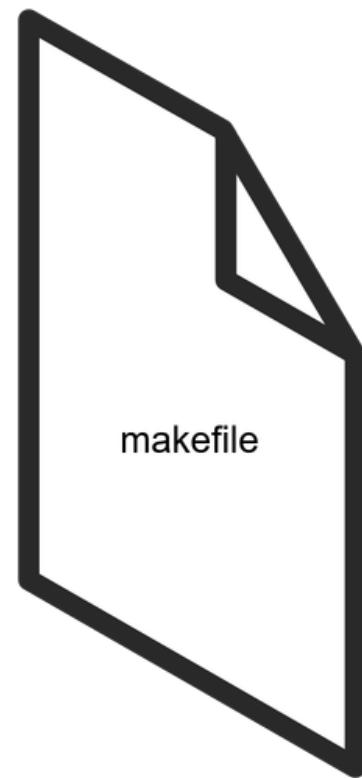
Transaction Verifications



Examples of corner cases considered when verifying the handshake signal of the cross bar transactions

Case 6 stalling is the most common case happening when the FUs are fully utilised. 45

Verification Scripting

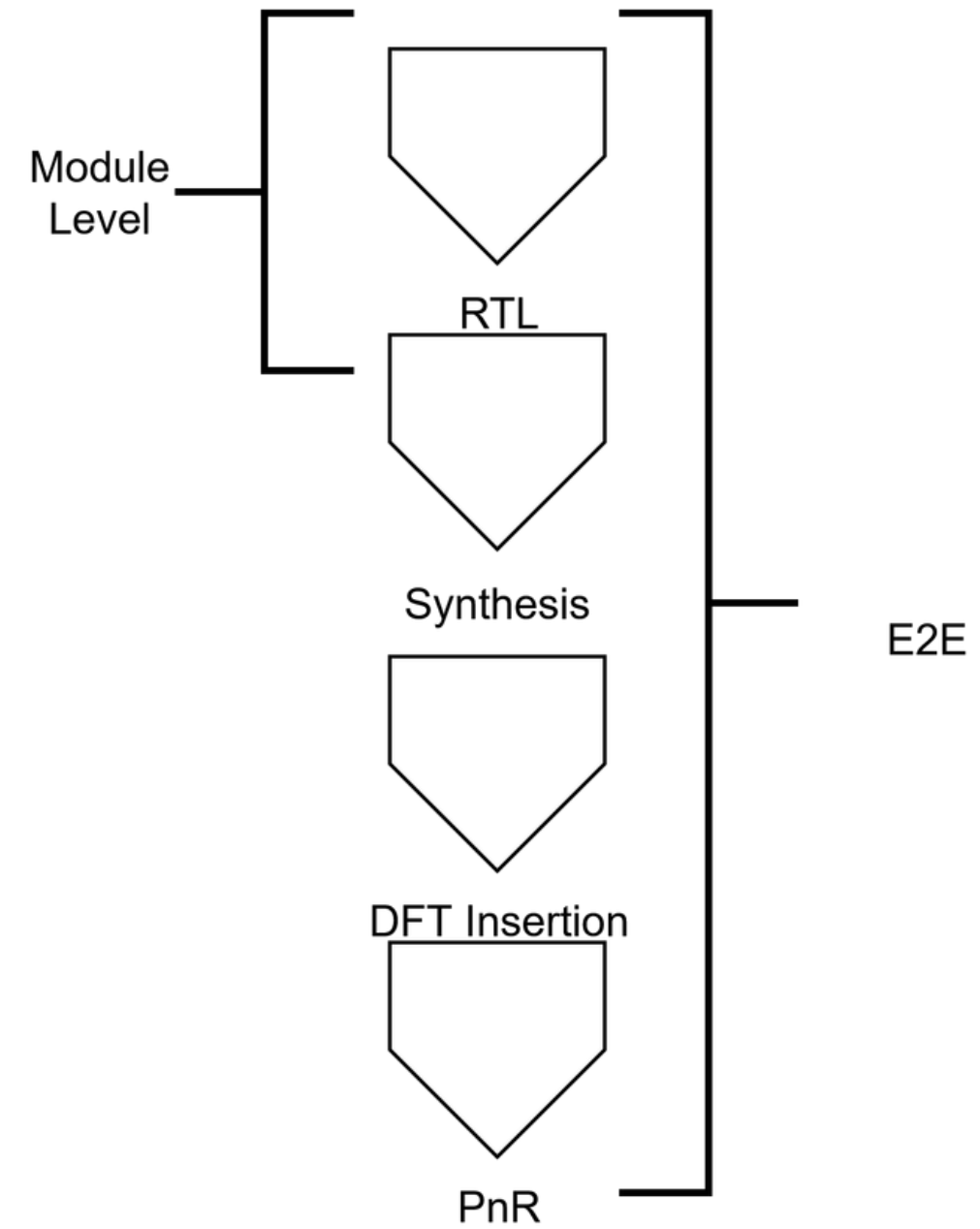


```
INFO: Stop testsuite 'FFD Testsuite' (@ 236000) (/Users/damien/.svut/example/ffd_testbench.sv:116)
- Warning number: 0
- Critical number: 0
- Error number: 0
- STATUS: 2/2 test(s) passed
```

jtag_driver_t
tdi
tdo
tms
tck

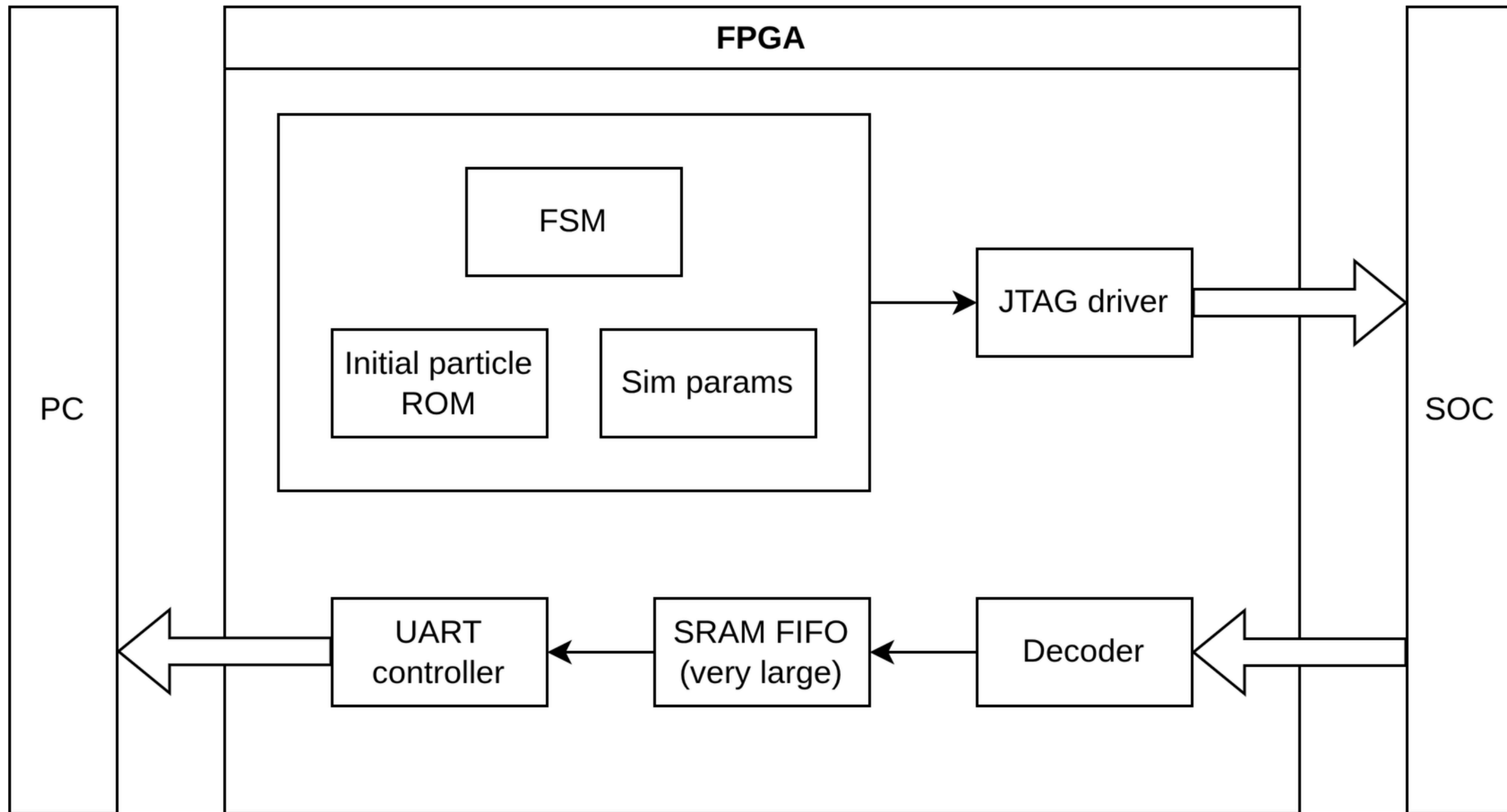
```
jtag_driver_t #(CHAIN_DR_LEN, CHAIN_IR_LEN) jtag_driver = new(jtag_if);
```

Verification Stages



Post tapeout Testing

Goal: Run same top level simulation on the taped out chip



Challenges

- Output tap is faster than UART can handle
 - need to buffer output in FPGA SRAM / Large FIFO
- Testing system entire system at once may not work
 - run smaller JTAG only tests if top test tails
- Manufacturing defects:
 - scan chains with ATPG (low overhead - all teams will need this tool)
- Testbench errors:
 - testbench/design co-simulation (possibly too much scope)
 - basic TB verification

Bring Up Steps

1. Structural bring-up

- Slow external clock, PLL bypassed – timing removed as a variable
- Scan-chain continuity + ATPG vectors (stuck-at coverage)

2. Control-path connectivity

- Still slow external clock
- JTAG scan_chain + IDCODE (3-TAP); single SRAM word write/readback

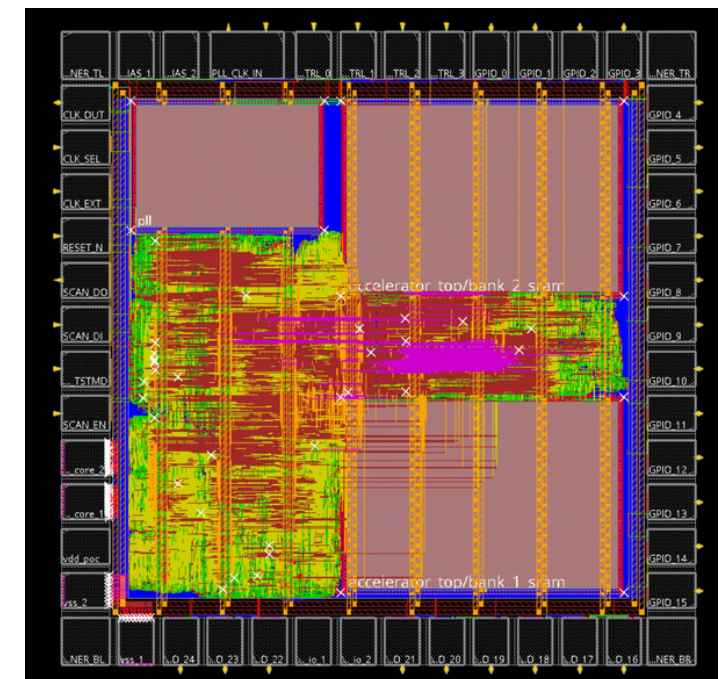
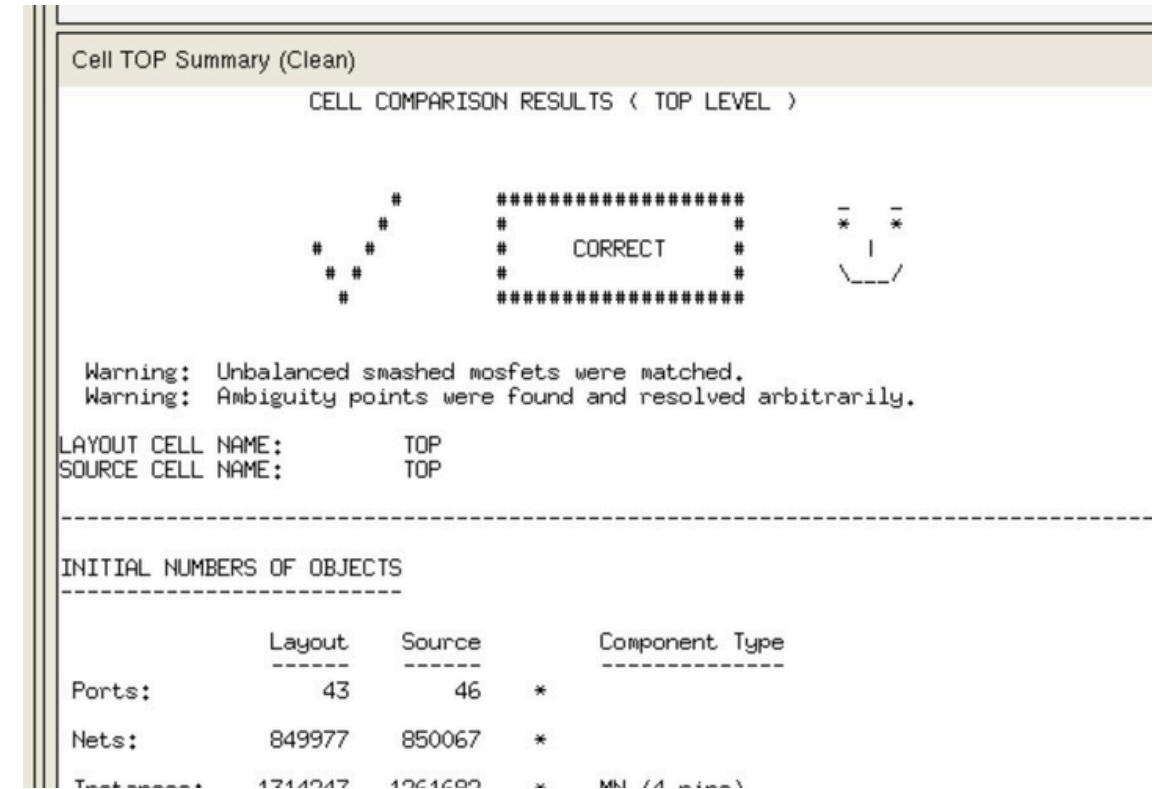
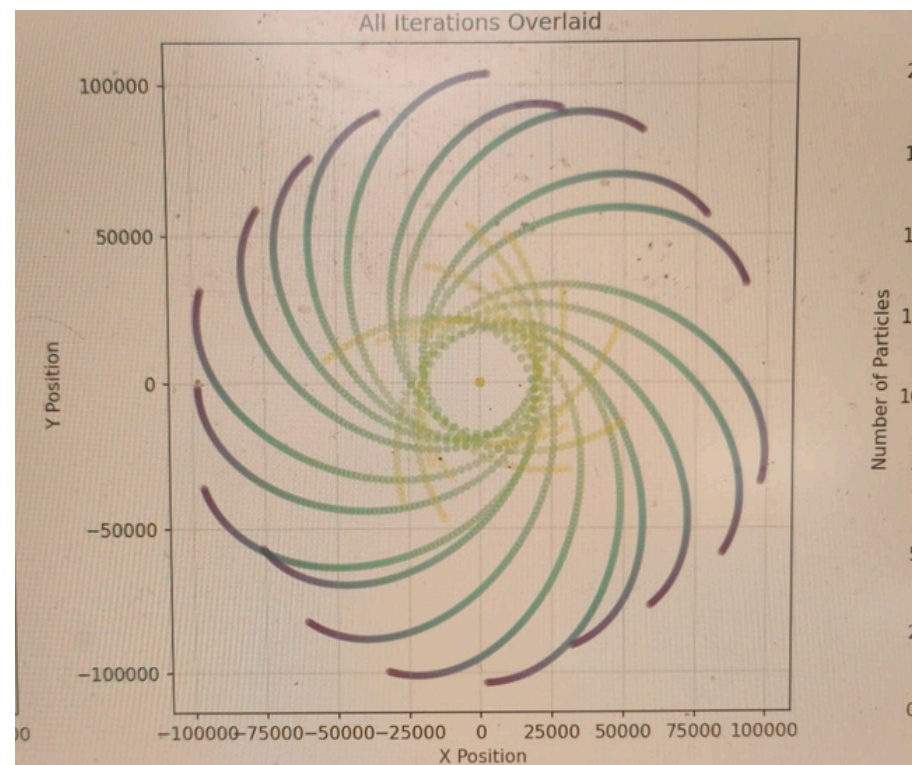
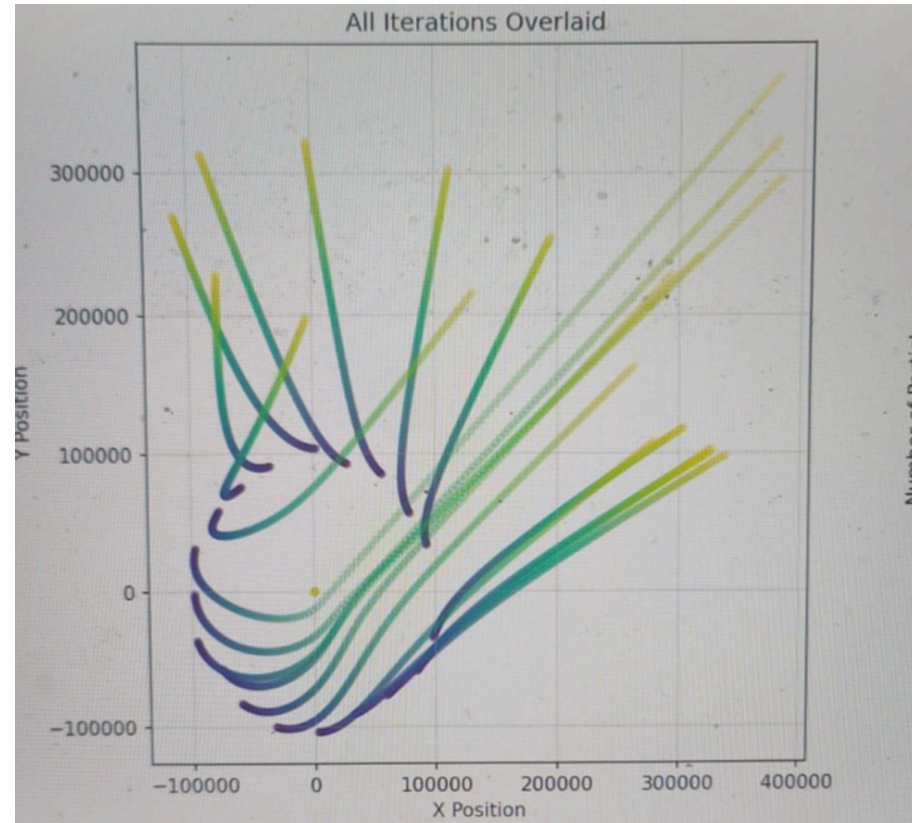
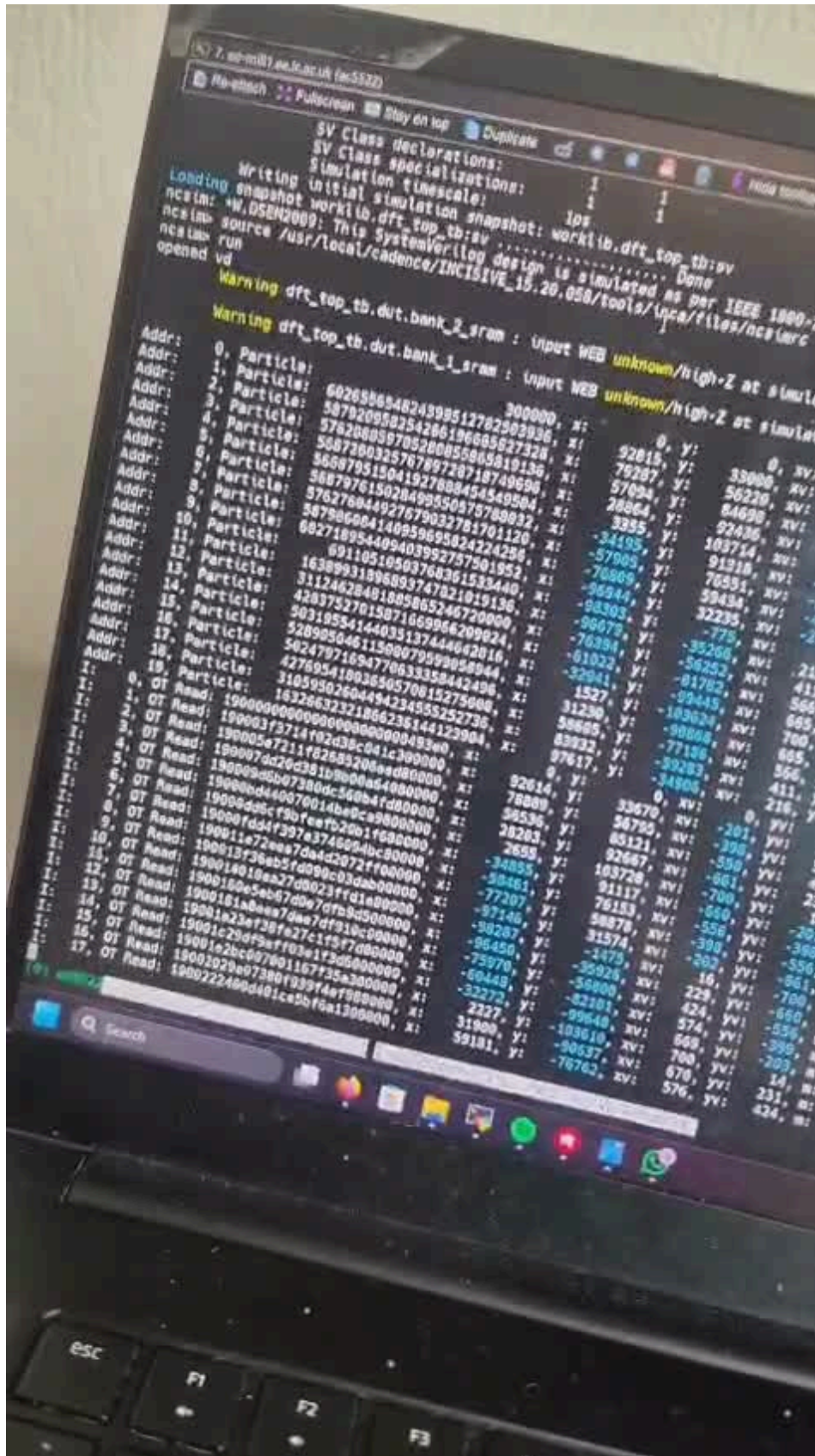
3. Minimal datapath

- PLL enabled at low frequency – Use the CDC
- 2-body scenario, 1 epoch → check one output-tap writeback vs model

4. Full functional at speed

- PLL ramped to 400 MHz – timing now under test
- Up to 2048 particles, many epochs, continuous stream vs C++ model

Results:



Lessons learned

- Following advice is a good idea:
 - scope conservatively, get something synthesizable ASAP
 - regularly examine synthesis estimates during development
 - leave lots of time for back end
- Generic hardware design principles work:
 - regressions / component tests save time, but doesn't ensure correctness
 - use the correct depth of SV features / conventions
 - have well defined design requirements
 - back of envelope calculations / performance optimisations

Thank You